COMSATS University Islamabad,

Sahiwal Campus, Pakistan.

CampusEase

A project presented to

COMSATS University Islamabad,

Sahiwal Campus

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022–2026)

By

M. Talha Nadeem khan	**CIIT/FA22-BCS-052/SWL**
Huzaifa Arshad	**CIIT/FA22-BCS-034/SWL**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any other degree or qualification at this or any other university or institute of learning.

M. Talha Nadeem khan
FA22-BCS-052

Huzaifa Arshad
FA22-BCS-034

CERTIFICATE OF APPROVAL

It is to certify that the Final Year Project of BS (CS) **"CampusEase"**: An Integrated Campus Operations and Assistance Platform was developed by **M. Talha Nadeem khan (CIIT/FA22-BCS-052)** and **Huzaifa Arshad (CIIT/FA22-BCS-034)** under the supervision of **"Mr. Bilal Shabir Qaisar"** and co-supervisor **"Mr. Ahmad Farid"**, and that in their opinion it is fully adequate in scope and quality for the degree of Bachelor of Science in Computer Science.

Supervisor

Mr. Bilal Shabir Qaisar

Lecturer

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Shahid Farid

Associate Professor

Department of Computer Science BZU Multan

Head of Department

Dr. Zafar Iqbal Roy

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **"Mr. Bilal Shabir Qaisar"** and our Co-Supervisor **"Mr. Ahmad Farid"**. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

M. Talha Nadeem khan
FA22-BCS-052

Huzaifa Arshad
FA22-BCS-034

Executive Summary

CampusEase is an AI-enhanced mobile and web platform developed as a Final Year Project to centralise three core campus operational services institutional announcements, lost and found management, and incident reporting at COMSATS University Islamabad, Sahiwal Campus, Pakistan. The platform is built on Flutter for the mobile application, React for the administrative web dashboard, Firebase as the cloud backend, OneSignal for push notification delivery, MobileNetV3 via TensorFlow Lite for on-device AI image similarity, and the Google Gemini API for incident triage. A supporting web platform deployed on Vercel serves as the public-facing entry point, APK distribution endpoint, deep link fallback layer, and password reset handler for the broader ecosystem. The system enforces a four-role access model (student, staff, office, and public) supplemented by three boolean lifecycle flags (active, verified, approved) through a three-layer authentication architecture, and implements a multi-tenant SaaS model that isolates data per institution via tenant_id scoping across all Firestore collections.

The Lost and Found module implements a six-stage peer-to-peer claim state machine with OTP-based physical handover verification, OTP recovery and meetup rescheduling controls, and a full audit trail, supported by on-device MobileNetV3 image embeddings and TF-IDF text similarity for automated match suggestions. The Announcements module enables targeted broadcast notifications delivered via a Firestore queue pattern relayed through OneSignal, while the Incident Reporting module provides tenant-configurable dynamic categories and AI-assisted triage via the Gemini API. A deep linking system supports shareable item and announcement links with guest preview fallback for unauthenticated users, and a hybrid password reset workflow combining the Firebase Admin SDK with Vercel serverless functions ensures reliable delivery to institutional email domains. The system was developed over four Agile sprints, validated through 26 functional requirements and a comprehensive test suite covering system, unit, functional, and integration testing, with all functional and non-functional requirements confirmed as met.

List of Abbreviations

The following abbreviations and acronyms are used throughout this report:

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CRUD	Create, Read, Update, Delete
FCM	Firebase Cloud Messaging
Firestore	Firebase Cloud Firestore (NoSQL database by Google)
Flutter	Google's open-source UI SDK for cross-platform mobile development
FYP	Final Year Project
HTTPS	Hypertext Transfer Protocol Secure
ITSM	IT Service Management
ML	Machine Learning
MobileNetV3	Mobile Neural Network Version 3 lightweight CNN architecture by Google
NoSQL	Non-Relational Database (Not Only SQL)
OTP	One-Time Password
RBAC	Role-Based Access Control
React	JavaScript library for building user interfaces (Meta/Facebook)
REST	Representational State Transfer (API architectural style)
SDK	Software Development Kit
TF-IDF	Term Frequency–Inverse Document Frequency (text similarity metric)

Abbreviation	Full Form
TFLite	TensorFlow Lite , lightweight ML inference framework for mobile devices
TOC	Table of Contents
UI	User Interface
UID	Unique Identifier
UML	Unified Modelling Language
URL	Uniform Resource Locator
UX	User Experience
WCAG	Web Content Accessibility Guidelines
YOLO	You Only Look Once (real-time object detection algorithm)

Table of Contents

List of Figures

List of Tables

CHAPTER 1

INTRODUCTION

1 Introduction

1.1 Brief

CampusEase is a mobile and web-based integrated platform engineered to modernise and streamline core campus operational services at university-level institutions. The system is targeted initially at COMSATS University Islamabad, Sahiwal Campus, Pakistan, and encompasses three tightly integrated modules: an Announcements Module, a Lost and Found Management System, and an Incident Reporting Module. By consolidating these services within a single digital ecosystem, **CampusEase** eliminates the operational silos that typically characterise campus administrative processes.

The platform's mobile application, developed using the Flutter framework for Android and iOS compatibility, serves as the primary interface for students and staff. An administrative web dashboard, built with React, provides institutional administrators with comprehensive oversight, moderation, and analytics capabilities. The backend is powered entirely by Firebase cloud services, encompassing authentication, real-time Firestore database operations, cloud storage, and push notifications via Firebase Cloud Messaging (FCM), supplemented by OneSignal for enhanced notification delivery. A key differentiating feature is the integration of MobileNetV3, a lightweight convolutional neural network, deployed on-device via TensorFlow Lite for AI-assisted image similarity matching within the Lost and Found module, alongside the Google Gemini API for AI-assisted incident category and priority triage. A supporting web platform deployed on Vercel further extends the ecosystem with a public-facing entry point, APK distribution, deep link fallback routing, and password reset handling.

The system enforces a role-based access model with four functional roles (student, staff, office, and public) supplemented by three boolean lifecycle flags (active, verified, and approved) that govern account access states independently of role assignment. This hierarchy supports both university campus environments, where students register via institutional email with roll number verification and staff accounts require admin approval, and city or district deployments, where general public users register freely. OTP-based handover verification ensures secure and verifiable transfer of claimed items, with tenant-configurable dynamic categories enabling institutions to define context-appropriate reporting taxonomies at runtime.

Architecturally, CampusEase is implemented as a multi-tenant platform. A single shared Firebase Firestore database serves all institutional deployments, with every document scoped by a tenant_id field that enforces complete data isolation at the query layer. This allows COMSATS University Islamabad, Sahiwal Campus the primary deployment target to operate alongside other campuses, cities, or district authorities under a single unified system, each with independent user bases, announcements, posts, and incidents.

1.2 Relevance to Course Modules

CampusEase draws substantively from multiple core modules of the Software Engineering and Computer Science curriculum, demonstrating the practical application of theoretical academic concepts:

Software Design: The system architecture follows established software design principles including modularity, separation of concerns, and layered architecture. The use of UML artefacts including use case diagrams, sequence diagrams, class diagrams, and ER diagrams demonstrates the application of formal design methodologies as taught in software design modules.

Database Systems: Firebase Firestore serves as a NoSQL cloud database with structured collections, access rules, and relational modelling via document references. The data design section of this report includes a fully specified data dictionary and entity-relationship diagram, directly applying concepts from database theory.

Mobile Application Development: The Flutter framework is employed to construct a cross-platform mobile application targeting Android, with a component-based UI architecture, stateful widget management, and integration with native device capabilities including camera access and push notification reception.

Web Development: The administrative dashboard is developed using React 18, applying concepts from web development including component-based UI architecture, client-side routing via React Router, real-time data binding through the Firebase JavaScript SDK, and responsive layout design using the CoreUI component library. The dashboard communicates directly with Firestore and implements role-based rendering, demonstrating practical application of modern frontend web development principles.

Artificial Intelligence: The AI module employs MobileNetV3, a deep convolutional neural network, for image feature extraction and cosine similarity-based matching. This draws directly upon concepts from AI and machine learning, including neural network architectures, feature vector representations, and similarity metrics.

Software Project Management: The project was managed using the Agile methodology with structured iterative sprints, Gantt chart-based scheduling, requirements traceability, and team coordination practices aligned with professional software project management principles.

1.3 Project Background

University campuses are complex operational environments characterised by large, transient student populations, high volumes of administrative communication, frequent incidents of misplaced property, and the daily occurrence of safety and maintenance-related incidents. Traditional mechanisms for managing these operational domains notice boards, email circulars, physical lost-and-found offices, and paper-based incident registers are fundamentally ill-suited to the pace, scale, and expectations of contemporary campus life.

At COMSATS University Islamabad, Sahiwal Campus, these limitations are acutely felt. Announcements distributed through informal channels such as class WhatsApp groups and posted physical notices are inconsistent, non-archival, and inaccessible to students outside the immediate recipient group. Lost items reported verbally to the security office or administrative staff are frequently never recovered due to the absence of systematic tracking, categorisation, or owner-matching. Incident reports, when submitted at all, follow unstructured, informal pathways that impede accountability and resolution tracking.

The motivation for **CampusEase** arises directly from this operational context. The project team observed these inefficiencies through lived experience and consultation with campus administrative staff. A digital, centralised platform capable of unifying announcements, lost-and-found management, and incident reporting under a single authenticated system was identified as the optimal intervention. The integration of AI-assisted image matching addresses a specific technical challenge in the lost-and-found domain: the difficulty of manually searching through large item databases for visually similar objects. The outcome is a platform that is not merely a digital migration of existing processes but a substantive re-engineering of campus operational workflows.

1.4 Literature Review

A structured review of existing literature and comparable systems was undertaken across four primary domains relevant to the **CampusEase** project.

1.4.1 Campus Information and Management Systems

Campus information systems have evolved from static web portals to dynamic, mobile-first platforms. Studies by Al-Rahmi et al. [1] and Kumar et al. [2] document the increasing adoption of mobile technology in Higher Education Institutions (HEIs) for delivering administrative and academic services [3] . Systems such as university Learning Management Systems (LMS) including Moodle and Blackboard demonstrate the viability of centralised digital platforms for campus communication. However, these systems are predominantly focused on academic delivery and do not address operational services such as property management or incident reporting. Research by Yeboah and Ewur [4] confirms that fragmented communication channels in universities contribute to information asymmetry and reduced institutional efficiency. The gap between academic-facing platforms and operational management systems remains largely unaddressed in the existing literature.

1.4.2 Lost and Found Management Systems

The domain of digital lost-and-found systems has received modest attention in academic literature. Zhou et al. [5] present an image-matching framework for lost and found items using a MobileNetV3-based photo matching network integrated with perceptual hashing algorithms, demonstrating the viability of lightweight CNN-based matching for real-world lost property retrieval. Tan and Chong [6] describe an effective university campus lost and found system, documenting the practical operational benefits of a web and mobile-based platform for student property recovery. However, a common limitation identified across existing systems is the absence of robust claim verification mechanisms; most systems rely on administrative manual review without cryptographic or OTP-based ownership confirmation, creating potential for fraudulent claims. Additionally, few systems integrate text-based and image-based similarity in combination, and those that do typically rely on server-side inference, introducing latency and privacy concerns that on-device processing resolves.

1.4.3 Incident Reporting Systems

Digital incident reporting systems are well-established in enterprise and public sector contexts but

remain underdeveloped within university campus environments. Existing enterprise solutions such as ServiceNow and Jira Service Management provide sophisticated ticketing and escalation workflows but impose significant cost and configurational overhead unsuitable for academic deployment. Research by Schafer et al. [7] on critical incident preparedness and response on post-secondary campuses highlights that the primary barriers to incident reporting among students are anonymity concerns and the perceived inefficacy of formal reporting channels. A mobile-first, low-friction reporting interface with transparent status tracking as implemented in **CampusEase** directly addresses these barriers. The system design draws upon established practices in IT service management (ITSM) for structured incident categorisation and routing.

1.4.4 AI-Assisted Image Recognition Systems

The application of convolutional neural networks (CNNs) for image similarity and retrieval has been extensively studied. Howard et al. [8] introduced MobileNetV3, demonstrating that highly efficient CNN architectures could achieve competitive accuracy on image classification benchmarks while remaining deployable on mobile hardware. Passalis and Tefas [9] propose a deep supervised hashing approach using quadratic spherical mutual information, establishing information-theoretic foundations for compact, efficient image embedding and retrieval directly relevant to the embedding-based similarity search employed in **CampusEase**. TensorFlow Lite documentation [10] confirms the feasibility of deploying MobileNetV3 on Android devices with acceptable inference latency. The primary limitation of existing AI-integrated lost-and-found solutions identified in the literature is the dependence on cloud-based inference APIs, which introduce network latency, data privacy risks, and operational costs. **CampusEase** addresses this by performing all image embedding generation on-device, transmitting only the resulting numerical embedding vectors to the Firestore database rather than the raw images for comparison purposes.

1.4.5 Comparative Analysis of Related Systems

The following table provides a structured comparison of existing related systems against **CampusEase**. It identifies the key features and limitations of each comparable system and demonstrates how CampusEase resolves or improves upon those shortcomings.

Table 1.1:Existing systems vs CampusEase improvements.

System / Platform	Key Features	Identified Weaknesses / Limitations	How CampusEase Addresses This
Moodle / Blackboard (LMS)	Centralised academic content delivery; role-based access; assignment management; discussion forums.	Exclusively academic in scope; no lost-and-found, incident reporting, or operational service management. Not designed for campus property or safety workflows.	CampusEase extends the digital campus concept to operational services announcements, lost-and-found, and incident reporting under a single authenticated platform, filling the gap LMS tools leave entirely unaddressed.
University WhatsApp / Email Groups	Familiar interface; instant delivery; zero setup cost.	Uncontrolled membership; no archival; no role enforcement; announcements easily missed or buried; no accountability or audit trail.	CampusEase provides a structured announcement broadcast system with tenant-scoped targeting, push notification delivery via OneSignal, Firestore persistence for retrospective access, and full admin control over content.
Physical Lost-and-Found Register (Manual)	No technology dependency; universally accessible.	No image documentation; no digital search; manual matching only; requires physical visit during office hours; no OTP or identity-verified claim process; high fraud risk.	CampusEase introduces AI-powered image similarity matching (MobileNetV3, cosine similarity ≥0.70), a six-stage OTP-verified peer-to-peer claim state machine, and Cloudinary-hosted photographic records eliminating every limitation of the manual register.
LostNet[5]	MobileNetV3-based image matching with perceptual hashing; server-side inference.	Server-side inference introduces network latency and data privacy risks; no claim verification workflow; no OTP handover; no multi-tenant support; academic prototype only.	CampusEase performs all embedding generation on-device (TFLite), preserving user privacy and removing network dependency. It adds a full claim state machine with OTP, meetup scheduling, dispute handling, and production-grade multi-tenant data isolation.

System / Platform	Key Features	Identified Weaknesses / Limitations	How CampusEase Addresses This
Tan & Chong University Lost-and-Found System[6]	Web and mobile platform for item reporting; basic search and filter; admin moderation.	No AI-based image matching; no OTP claim verification; manual admin review for all claims; no push notification system; no multi-institution support.	CampusEase automates match discovery through AI embeddings and TF-IDF text similarity, introduces peer-to-peer OTP handover, and supports multiple institution types (campus, city, district) under one deployment.
ServiceNow / Jira Service Management (Enterprise ITSM)	Sophisticated ticketing, escalation workflows, SLA management, analytics dashboards.	Prohibitively expensive for academic deployment; high configurational overhead; not designed for student-facing mobile use; no AI image matching; no campus-specific workflows.	CampusEase delivers structured incident categorisation, status tracking, push notification on every status change, and AI-assisted triage (Google Gemini API for auto-priority and category suggestion) at zero licensing cost, optimised for campus mobile users.
Generic Incident Reporting Apps (e.g., FixMyStreet)	Public incident submission; location tagging; status visibility.	Not authenticated; no role-based routing; no push notifications to submitter; no multi-tenant isolation; no campus-specific categories or admin workflow integration.	CampusEase enforces Firebase Authentication [11] , routes incidents by category to the correct authority, delivers targeted push notifications at every status transition, and isolates all data by tenant_id to prevent cross-institution data exposure.

1.5 Analysis from Literature Review

The literature review yielded several direct design influences that shaped the architecture and feature set of CampusEase:

First, the absence of OTP-based handover verification in existing lost-and-found systems was identified as a critical gap. CampusEase introduces OTP verification as a mandatory step in item handover, ensuring that only the verified owner can collect a claimed item. This design decision was directly motivated by the fraud-vulnerability identified in reviewed systems.

Second, the privacy and latency limitations of server-side AI inference, documented in several reviewed works, led to the adoption of on-device MobileNetV3 inference via TensorFlow Lite. This architectural choice preserves user privacy by keeping raw images local and reduces the system's dependency on network availability for AI operations.

Third, the documented communication fragmentation in campus environments [3] informed the decision to build a unified multi-module platform rather than isolated point solutions. A single authenticated application reduces friction for end users and ensures consistent notification delivery across all operational domains.

Fourth, the recognition that text-based similarity in combination with image-based matching improves retrieval accuracy [5] led to the inclusion of TF-IDF cosine similarity for item description matching as a complementary mechanism to visual embedding comparison.

Finally, the barriers to campus incident reporting identified by Schafer et al. [7] directly informed the UI/UX design philosophy of the Incident Reporting Module, which prioritises minimal input steps, optional anonymity, and visible status feedback to encourage reporting uptake.

Sixth, the literature on flag-based state management and scalable RBAC architectures directly informed the decision to implement a simplified four-role system (student, staff, office, public) supplemented by boolean state flags (active, verified, approved) rather than creating multiple role variants for transient conditions. This design reduces administrative complexity, improves query efficiency through composite Firestore indexes on role-flag combinations, and aligns the system with industry-standard IAM patterns used in enterprise and government authentication frameworks. The three-layer authentication model implemented in CampusEase, comprising identity check, access state verification, and role-based routing, directly operationalises the defence-in-depth principles identified in the reviewed literature.

1.6 Methodology and Software Lifecycle for This Project

The development of CampusEase followed the Agile software development methodology [12] combined with an iterative development model. This dual-framework approach provided the flexibility and structured incrementalism necessary to accommodate the project's multifaceted technical requirements, including AI integration, cloud backend services, multi-platform frontend development, serverless infrastructure, and an administrative web panel.

The project was structured into four primary iterations, each delivering a functional and testable increment of the system

- **Iteration 1: Authentication and Core Reporting**

Authentication and Core Infrastructure: Established Firebase Authentication with OTP-based email verification, structured Firestore collections with tenant_id scoping, Flutter project scaffolding, and foundational lost and found item reporting with Cloudinary image upload[13].

- **Iteration 2: Search, Governance, and Admin Panel**

Implemented category-based search and filtering, user history tracking, Firestore query optimisation, and the React admin dashboard with role-based access, staff and extended student approval workflows, session management, and multi-tenant switching.

- **Iteration 3: Claims, Incidents, and Announcements**

Developed the full six-stage peer-to-peer claim state machine with OTP handover verification, the Incident Reporting module, and the Announcements module with targeted push notification delivery via the OneSignal queue pattern.

- **Iteration 4: AI Integration and Extended Features**

Integrated the MobileNetV3 on-device image similarity engine, Google Gemini API for incident triage, deep linking with guest preview support, password reset via Vercel serverless functions, dynamic tenant-configurable categories, and the CampusEase web platform.

Sprint ceremonies including sprint planning, daily stand-ups, and retrospective reviews were conducted iteratively throughout the project lifecycle. Requirements were maintained in a living backlog and refined progressively as implementation revealed additional technical constraints and opportunities.

1.6.1 Rationale behind Selected Methodology

Agile was selected over traditional waterfall or V-model methodologies for several reasons specific to the nature of this project. The involvement of an AI module with an empirically determined similarity threshold introduced inherent uncertainty into the requirements; the Agile principle of responding to change over following a plan was therefore operationally appropriate. Similarly, the integration of multiple third-party services including Firebase, OneSignal, MobileNetV3 via TensorFlow Lite [14],Google Gemini API, EmailJS, and Vercel serverless functions required

iterative verification and adjustment that a fixed-scope waterfall process would have accommodated poorly.

The iterative model complemented Agile by ensuring backward compatibility between increments. Each iteration's deliverable was a working system increment rather than a design artefact, enabling continuous integration testing and stakeholder review. The combination of Agile adaptability with iterative structure provided a governance framework that balanced flexibility with the academic accountability requirements of an FYP project, ensuring milestones, deliverables, and documentation remained on schedule throughout the development lifecycle.

CHAPTER 2

PROBLEM DEFINITION

2 Problem Definition

2.1 Problem Statement

University campus environments generate high volumes of operational service interactions daily ranging from administrative announcements and lost property reports to safety incidents yet the systems used to manage these interactions at most institutions, including COMSATS University Islamabad, Sahiwal Campus, remain predominantly manual, informal, and fragmented. Specifically, the following problems have been identified:

- **Announcement Inaccessibility** Campus announcements are disseminated through multiple uncontrolled channels including physical notice boards, email lists, and informal messaging groups. This results in inconsistent information reach, inability to verify receipt, and an absence of archival access for students who require retrospective information.

- **Inefficient Lost and Found Management** Lost and found items reported to the campus security office are tracked manually without categorisation, image documentation, or AI-assisted matching. Recovery rates are low due to the inability to systematically associate reported lost items with found items, and the claim process lacks identity verification, creating opportunities for fraudulent retrieval.

- **Absence of Structured Incident Reporting** Campus incidents including maintenance failures, safety concerns, and behavioral incidents are reported through ad hoc verbal or email channels without structured categorisation, status tracking, or administrative accountability. This impedes timely resolution and prevents pattern analysis for proactive campus safety management.

- **Lack of Role-Based Workflow Integration** Existing approaches treat each operational domain independently, requiring students and staff to engage with separate, disconnected systems. There is no unified platform that enforces appropriate access control, routes service requests to the correct authorities, and provides transparent status feedback to reporting users.

CampusEase directly addresses each of these problems through a unified, authenticated, AI-enhanced platform that systematises campus announcements, automates lost-and-found item matching, and provides a structured incident reporting workflow with role-based access and transparent status management.

2.2 Deliverables and Development Requirements

The following deliverables were defined at project initiation and refined through iterative development:

Primary Deliverables

- **Flutter Mobile Application (Android/iOS initial install size: 30–50 MB)**

Providing student, staff, office, and public users access to all three modules. Campus students authenticate via institutional email with roll number and email verification; public users (city/district tenants) register with any email; all data is scoped to the user's tenant.

- **React Admin Web Dashboard (React 18 + CoreUI)**

Providing super admins and tenant admins with moderation, staff approval, tenant management, announcement creation, analytics, dispute resolution, and AI match review.

- **Firebase Backend Infrastructure**

Firestore (NoSQL database with tenant-scoped security rules), Firebase Authentication (primary + secondary app instance for staff creation), Cloud Storage, and Firebase Cloud Messaging. No Firebase Cloud Functions are used; all notification dispatch is handled by the Flutter NotificationService, and all AI similarity computation is performed on-device within the Flutter application.

- **AI Image Similarity Module**

MobileNetV3 model integrated via TensorFlow Lite for on-device image embedding generation, with cosine similarity computed against Firestore-stored embeddings.

- **OneSignal Notification Integration**

For enhanced push notification delivery to mobile devices beyond the constraints of the Firebase free tier.

2.3 Current System

Announcement Distribution

Administrative and academic announcements are currently distributed via departmental email lists, physical notice boards located at campus entry points and departmental buildings, and informal class representative WhatsApp groups. No central archive exists; announcements are routinely missed by

students not in the correct group or physically absent from campus on the day of posting. There is no mechanism to confirm receipt or engagement.

Lost and Found Handling

Found items on campus are submitted to the security office or administrative block, where they are logged in a physical register with minimal categorisation (typically item type and date found). Students seeking lost items must physically visit the security office during working hours to manually search the register and inspect stored items. No photographic record is maintained, no digital database exists, and the item matching process is entirely reliant on verbal descriptions. Claim verification is performed by security staff based on subjective assessment of the claimant's knowledge of the item; no OTP or formal verification protocol is in use.

Incident Reporting

Campus incidents are currently reported verbally to faculty members, security staff, or the administrative office, depending on the nature of the incident. No standardised incident reporting form exists. There is no digital tracking system, meaning incidents cannot be categorised, escalated, or resolved with accountability. Follow-up on reported incidents is entirely at the discretion of the receiving staff member, and the reporting student receives no status feedback.

CHAPTER 3

REQUIREMENT ANALYSIS

3 Requirement Analysis

3.1 Use Case Diagram(s)

The CampusEase functional model is defined by the interaction between three core actor categories and the system boundary. The following diagram illustrates the relationship between these actors and the platform's modular capabilities.

Actor Definitions and Interactions:

- **User (Student / Public):** This combined actor represents the primary consumer of the application. While their registration logic differs Students requiring OTP and semester-based approval while Public users receive immediate access their functional capabilities within the app are identical. They interact with core modules to report items/incidents, submit claims via **OTP verification**, and track the status of their requests in real-time.

- **Staff:** A specialized campus actor restricted to the "Lost and Found" domain. Staff members focus on high-volume item management and assisting students with handover processes. Following a secure architectural boundary, they are excluded from the Incident Reporting and Announcement modules to maintain administrative separation.

- **Admin (Super / Campus):** The governing actor who operates via the React Dashboard. The Admin manages the system's "source of truth," overseeing tenant-level settings, viewing global analytics, and moderating reports. They serve as the final decision gate for user approvals and the primary source for broadcasting multi-tenant announcements.

System Logic & Relationships

The use cases are structured to ensure data integrity and security. For example, the "Submit Claim" use case includes "OTP Verification" as a mandatory sub-process to prevent unauthorized handovers. Similarly, administrative tasks like "Manage Tenants" are strictly scoped to the Super Admin role, ensuring that institution-specific data remains isolated and secure within the multi-tenant framework.

This use case model **Figure 3.1** ensures clear role separation, secure interactions, and a well-structured workflow across all core functional modules of the CampusEase system.

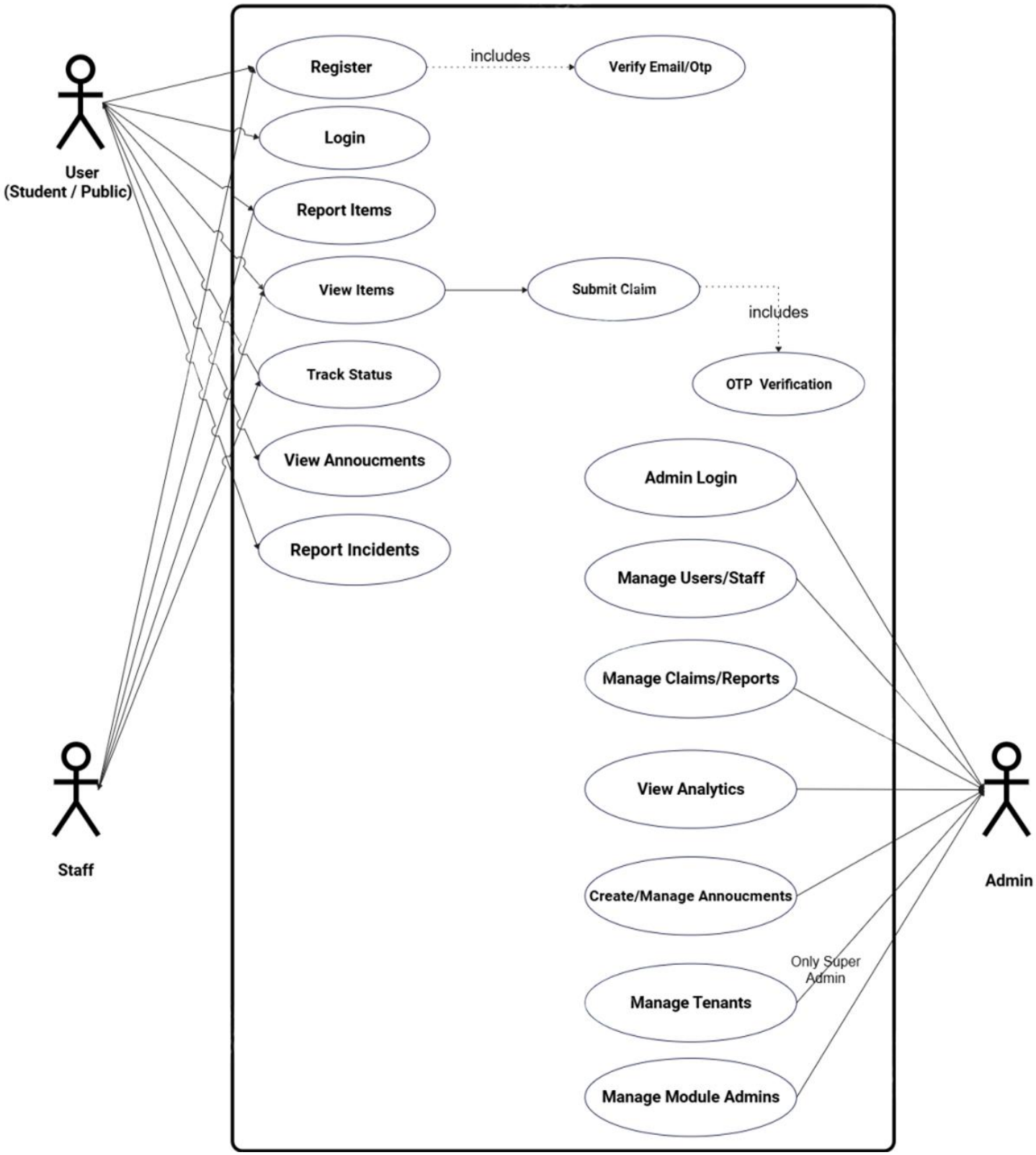

Figure 3.1:Use Case Diagram

3.2 Detailed Use Case Descriptions

The following tables present detailed use case specifications for the primary functional interactions within CampusEase.

Table 3.1:Use Case UC-01 User Registration and Authentication

Field	Description
Use Case ID	UC-01
Use Case Name	User Registration and Authentication
Actors	Primary: Student, Office Staff, Administrator; Secondary: Firebase Authentication System
Description	A user registers an account using their university email address, receives OTP verification, and subsequently logs into the system to access role-specific features.
Preconditions	User possesses a valid university email address. Firebase Authentication service is operational.
Postconditions	User account is created in Firestore with appropriate role assignment. User is authenticated and redirected to their role-specific home dashboard.
Main Flow	1. User opens the CampusEase mobile application. 2. User selects Register and enters their university email and desired password. 3. System creates account in Firebase Authentication and writes user document to Firestore with role, tenant_id, and initial flags (active=true, verified=false, approved based on semester). 4. System sends OTP to the provided email via EmailJS. 5. User enters OTP on the verification screen. 6. User is redirected to the Home Dashboard. 7. For subsequent logins: User enters credentials; system validates through the three-layer authentication process (identity check, flag-based access state verification, role-based routing) and routes to the appropriate application shell.

Field	Description
Alternative Flows	AF-1: If email domain is not university-approved, system rejects registration. AF-2: If OTP expires, user may request a new OTP. AF-3: If credentials are invalid at login, system displays error and allows retry.
Exceptions	E-1: Firebase service unavailability prevents authentication operations. E-2: Network failure during OTP transmission.

Table 3.2:Use Case UC-02: Report Item (Lost or Found)

Field	Description
Use Case ID	UC-02
Use Case Name	Report Item (Lost or Found)
Actors	Primary: Student, Staff, Public User; Secondary: Cloudinary, AI Module (MobileNetV3)
Description	An authenticated user reports a lost or found item by completing a structured form. The user selects whether the item is lost or found via a toggle. The system stores the report, uploads the image to Cloudinary, generates an on-device AI embedding, and initiates similarity matching against items of the opposite type.
Preconditions	User is authenticated. MobileNetV3 model is loaded on device. Camera or gallery access is granted.
Postconditions	Item report is stored in Firestore with type set to lost or found. Image is uploaded to Cloudinary and URL saved. 1280-dimensional embedding is generated and stored. Cosine similarity matching is triggered against all opposite-type items in the same tenant.
Main Flow	1. User navigates to Report Item and selects Lost or Found via the form toggle. 2. User completes the form: title, description, category, location, date, and optional ownership questions (for found items). 3. User uploads an item image via camera or gallery. 4. Image is uploaded to Cloudinary; the returned

Field	Description
	HTTPS URL is stored in the Firestore post document. 5. MobileNetV3 (TFLite) generates a 1280-dimensional embedding on-device from the uploaded image. 6. Embedding is written to the Firestore post document. 7. PostsProvider retrieves embeddings of all items of the opposite type from the same tenant and computes cosine similarity on-device for each pair. 8. All pairs with similarity score >= 0.70 are written to the match_suggestions collection. 9. User receives a push notification if matches are found; match suggestions are displayed on the item detail screen.
Alternative Flows	AF-1: If image upload to Cloudinary fails, user is prompted to retry before submission proceeds. AF-2: If the AI module is unavailable or inference times out, the report is stored without an embedding and matching is deferred until the next successful submission. AF-3: If the user selects Found, the form additionally prompts for ownership verification questions that the true owner must answer during the claim process.
Exceptions	E-1: Firestore write failure prevents report storage; user is shown an error and no embedding is generated. E-2: Insufficient device memory for MobileNetV3 inference; report is stored without embedding.

Table 3.3:Use Case UC-03 Claim Item: Multi-Stage Peer Verification

Field	Description
Use Case ID	UC-03
Use Case Name	Claim Item Multi-Stage Peer Verification Workflow
Actors	Primary: Student (Claimant), Student or Office Staff (Holder/Post Owner); Secondary: Firebase Firestore, OtpProvider, NotificationService, AuditService
Description	A student submits a claim against a post. The workflow is a six-stage, peer-to-peer state machine governed by the post holder and claimant. The admin is only involved if a dispute is raised. Two claim actions exist: action=claim

Field	Description
	(claiming a Found post) and action=found (reporting they found a Lost post). Both follow the identical status machine.
Preconditions	Post exists in Firestore with status active. Authenticated user is not the post owner.
Postconditions	Claim document reaches status completed. Post status updated to resolved. Audit event handoverComplete logged. Both parties notified.
Main Flow	Stage 1 Submission (pending): Claimant fills ClaimOrFoundFormPage and submits. ClaimProvider.createClaim() executes a Firestore batch write: claim document, user_claims_index entry, and audit event. Push notification sent to holder. Stage 2 Holder Review: Holder reviews details in HolderClaimManageScreen. Accept: batch write sets claim=accepted, post=under_claim, primaryClaimId. Reject: batch write sets claim=rejected with optional reason; post reverts to active. Stage 3 Meetup Scheduling: Holder proposes meetup via MeetupProvider (meetup_proposed). Claimant accepts (meetup_confirmed) or counter-proposes. Loop until both confirm. Stage 4 OTP Handover: Holder generates 6-digit OTP (otp_pending). OTP displayed holder-side only. Claimant enters OTP; verifyOtp() validates (handover_pending). Holder confirms handover (completed; post=resolved). AuditService logs handoverComplete. Both parties notified.
Alternative Flows	AF-1: Holder rejects post reverts to active. AF-2: Claimant counter-proposes meetup loops at Stage 3. AF-3: Claimant cancels (permitted from pending, accepted, meetup_proposed, meetup_confirmed; blocked from otp_pending onward). AF-4: Either party raises dispute status=disputed, dispute document written to disputes collection for admin review. AF-5: Office staff reuse HolderClaimManageScreen with no duplicate logic.
Exceptions	E-1: Firestore batch atomicity prevents partial state. E-2: OtpProvider throws if verifyOtp called before status=otp_pending. E-3: _assertOwner() blocks non-owners from holder actions. E-4: Multiple claimants each has an independent claim; only one can be primaryClaimId.

Table 3.4:Use Case UC-04: Report Incident

Field	Description
Use Case ID	UC-04
Use Case Name	Report Incident
Actors	Primary: Student, Staff, Public User; Secondary: Administrator, Google Gemini API, NotificationService
Description	An authenticated user reports an incident such as a safety concern, maintenance issue, or behavioural matter via a structured form. The system invokes the Google Gemini API to automatically suggest an incident category and priority level, stores the report in Firestore scoped to the user's tenant, and notifies the relevant administrator via the OneSignal push notification queue.
Preconditions	User is authenticated. Incident has occurred on campus or within the user's tenant jurisdiction.
Postconditions	Incident report stored in Firestore with status set to pending. AI-suggested category and priority stored as ai_category and ai_priority fields. Push notification queued to relevant administrator via pendingPushNotifications collection. Status is visible to the reporting user in the My Incidents screen.
Main Flow	1. User navigates to Report Incident. 2. User selects incident category (Safety, Maintenance, Behavioural, Other) and provides a description, location, and optional image. 3. User optionally enables the anonymity toggle; if enabled, reporterId is set to null in the stored document while routing information is preserved. 4. On submission, the Flutter app calls the Google Gemini 2.5 Flash Lite API with the incident description; Gemini returns a suggested category and priority level (low, medium, or high) as a structured JSON response. 5. Report is written to Firestore with status=pending, the user-provided fields, and the AI-generated ai_category and ai_priority fields stored alongside them. 6. A document is written to the

Field	Description
	pendingPushNotifications collection targeting the relevant administrator. 7. Flutter NotificationService detects the pending document and calls the OneSignal REST API to deliver the push notification to the administrator. 8. Administrator reviews the report in the React admin dashboard, where both user-provided and AI-suggested classifications are displayed. 9. Administrator updates the status (reviewed or resolved); a push notification is dispatched to the reporting user at each status change. 10. Status updates are visible to the reporting user in real time via the My Incidents screen.
Alternative Flows	AF-1: Gemini API is unavailable or times out (default threshold: 10 seconds) report is stored with user-provided values only; ai_category and ai_priority are left empty and triage is marked as deferred. AF-2: Image upload fails report is submitted without an image attachment; all other fields are stored normally.
Exceptions	E-1: Firestore write failure prevents report storage; user is shown an error message and no notification is queued. E-2: OneSignal delivery fails report is still stored in Firestore and visible to the administrator in the React dashboard.

Table 3.5:Use Case UC-05 Broadcast Announcement

Field	Description
Use Case ID	UC-05
Use Case Name	Broadcast Announcement
Actors	Primary: Administrator; Secondary: OneSignal Notification Service
Description	An administrator creates and publishes a campus announcement via the React admin dashboard. The announcement is broadcast to all registered users via push notification and stored for in-app viewing.
Preconditions	Administrator is authenticated in the admin dashboard. OneSignal service is operational.

Field	Description
Postconditions	Announcement stored in Firestore. Push notification delivered to all registered mobile devices. Announcement visible in all users' in-app notification feed.
Main Flow	1. Admin navigates to Announcements in the admin dashboard. 2. Admin creates announcement with title, body, optional image, and priority. 3. Admin publishes. 4. Firestore announcement document created; React also writes a pending document to the pendingPushNotifications collection. 5. Flutter NotificationService (running on an authenticated device) detects the pending document, calls the OneSignal REST API directly, and marks the document as sent. 6. All registered users receive push notification. 7. Announcement also appears in the in-app notification feed via Firestore real-time listener.
Alternative Flows	AF-1: Admin may schedule announcement for future delivery. AF-2: Admin may target announcement to specific roles.
Exceptions	E-1: OneSignal delivery failure; announcement remains in Firestore and visible in-app.

Table 3.6:Use Case UC-06 View Announcements

Use Case ID	UC-06
Use Case Name	View Announcements
Actors	Primary: Student, Staff, Public User; Secondary: Firebase Firestore
Description	Authenticated users view campus or city/district announcements broadcast by administrators. Announcements are filtered by the user's tenant and displayed in reverse chronological order.
Preconditions	User is authenticated. Announcements exist in Firestore for the user's tenant.
Postconditions	Announcements are displayed. User may bookmark an announcement to their saved_announcements subcollection.
Main Flow	1. User navigates to the Announcements tab. 2. AnnouncementProvider streams all announcements for the user's tenant_id in real time from Firestore. 3. Announcements are displayed as cards with title, date, type badge, and pinned indicator. 4. User taps an

	announcement to view full details, including attachments. 5. User may bookmark the announcement for offline access.
Alternative Flows	AF-1: No announcements exist for the tenant, empty state displayed. AF-2: User is offline cached Firestore data is shown via Firestore native persistence.
Exceptions	E-1: Firestore read failure prevents announcement stream from loading.

Table 3.7:Use Case UC-07 Track Status

Use Case ID	UC-07
Use Case Name	Track Status
Actors	Primary: Student, Staff; Secondary: Firebase Firestore, NotificationService
Description	Users track the real-time status of their submitted lost/found item reports, claims, and incident reports. Status changes trigger push notifications to keep users informed without requiring manual refresh.
Preconditions	User is authenticated. At least one item report, claim, or incident report has been submitted by the user.
Postconditions	User views current status of all their submissions. Push notifications are received when status changes.
Main Flow	1. User navigates to My Reports or My Claims tab. 2. PostsProvider and ClaimProvider load all submissions associated with the authenticated user's UID, filtered by tenant_id. 3. Each item displays its current status badge (active, under_claim, resolved for posts; pending, accepted, meetup_proposed, meetup_confirmed, otp_pending, handover_pending, completed, rejected, cancelled, disputed for claims; pending, reviewed, resolved for incidents). 4. Firestore real-time listeners update statuses automatically when an admin or counterpart performs an action. 5. Push notification is received via OneSignal when status changes, directing user to the relevant screen.
Alternative Flows	AF-1: No submissions exist empty state shown with prompt to report an item. AF-2: Claim status is disputed dispute details and admin review indicator shown.
Exceptions	E-1: Firestore listener fails user must manually refresh. E-2: Push notification not delivered status still updates via Firestore listener on next app open.

3.3 Functional Requirements

The following table summarises the functional requirements of CampusEase.

Table 3.8:Functional Requirements

ID	Requirement	Description
FR-01	**User Authentication**	System shall support registration and login via Firebase Authentication using university email with OTP verification. High Priority.
FR-02	**Lost Item Reporting**	System shall allow authenticated users to submit lost item reports with title, description, category, location, date, and image upload. High Priority.
FR-03	**Found Item Reporting**	System shall allow authenticated users to submit found item reports with equivalent fields. High Priority.
FR-04	**AI Image Similarity**	System shall process item images using MobileNetV3[15] to generate 1280-dimensional embeddings and compute cosine similarity for match suggestions (threshold >= 0.70). Medium Priority.
FR-05	**Text-Based Similarity**	System shall compute TF-IDF cosine similarity between item descriptions to supplement image-based matching. Medium Priority.
FR-06	**Claim / Found Report Submission**	System shall allow authenticated users to submit a claim (action=claim on a Found post) or found report (action=found on a Lost post), completing ClaimOrFoundFormPage with contact details, date/time, location, proof image, and ownership question answers. High Priority.
FR-07	**Claim State Machine**	The claim workflow shall enforce a sequential status machine: pending, accepted, meetup_proposed, meetup_confirmed, otp_pending, handover_pending, completed (plus rejected, cancelled, disputed). All transitions shall be validated by ClaimProvider before any Firestore write. High Priority.

ID	Requirement	Description
FR-08	**Holder Accept / Reject**	The post holder shall be able to accept or reject a pending claim via HolderClaimManageScreen. All status changes shall be performed as atomic Firestore batch writes updating the claim, the parent post, and user_claims_index simultaneously. High Priority.
FR-08b	**Meetup Scheduling**	After acceptance, the holder shall propose a meetup time and location via MeetupProvider. The claimant may accept or counter-propose until both confirm. High Priority.
FR-08c	**OTP Handover Verification**	At meetup, the holder shall generate a 6-digit OTP displayed only on their screen. The claimant shall enter the OTP on their device. Following successful entry, the holder confirms physical handover, advancing claim to completed and post to resolved. High Priority.
FR-08d	**Dispute Handling**	Either party shall be able to raise a dispute or report a no-show from HolderClaimManageScreen. Status advances to disputed and a document is written to the disputes collection for admin review. Medium Priority.
FR-09	**Incident Reporting**	System shall provide a structured incident report form with category, description, location, and optional image. The system shall additionally invoke the Google Gemini 2.5 Flash Lite API to generate AI-suggested category and priority classifications, stored as ai_category and ai_priority fields, to assist administrative triage. High Priority.
FR-10	**Incident Status Tracking**	System shall allow reporting users to view the current status of submitted incidents. High Priority.
FR-11	**Announcement Creation**	Administrators shall be able to create, schedule, and broadcast announcements to all users or targeted role groups. High Priority.

ID	Requirement	Description
FR-12	**Push Notifications**	System shall deliver real-time push notifications for AI matches, claim updates, incident updates, and announcements via OneSignal and FCM. High Priority.
FR-13	**Search and Filter**	System shall allow users to search and filter items by category, date range, keywords, and status. Medium Priority.
FR-14	**User History**	System shall maintain a viewable history of each user's submitted reports, claims, and their statuses with timestamps. Medium Priority.
FR-15	**Admin Analytics**	Administrators shall access real-time analytics dashboards showing system activity, claim patterns, AI match accuracy, and incident trends. Medium Priority.
FR-16	**Role-Based Access Control**	System shall enforce distinct functional permissions for Student, Office Staff, and Administrator roles. High Priority.
FR-17	**Admin User Management**	Administrators shall be able to view, update roles, and deactivate user accounts. High Priority.
FR-18	**Privacy Control**	System shall ensure report data and match notifications are accessible only to involved users and administrators. High Priority.
FR-19	**Multi-Tenant Data Isolation**	All Firestore collections shall store a tenant_id field. Every read and write operation shall be scoped to the authenticated user's tenant_id, ensuring complete data isolation between institutions. High Priority.
FR-20	**Tenant-Aware Registration**	The registration flow shall validate the user's email domain against the active tenant's allowedDomain and studentDomain fields. High Priority.
FR-21	**Staff Approval Workflow**	Staff applicants with approved=false shall be blocked from accessing the main application until an administrator approves

ID	Requirement	Description
		their account. On approval, the approved flag is set to true and an EmailJS notification is dispatched to the staff member. High Priority.
FR-22	**Login Gate System**	The login flow shall enforce three sequential authentication layers: (1) identity verification: user document must exist in Firestore for the authenticated UID; (2) access state verification: active flag must be true, verified flag must be true for students (unless whitelisted), and approved flag must be true; (3) role-based routing: user is directed to the appropriate application shell based on their role field. High Priority.
FR-23	**Tenant Management**	Super administrators shall be able to create, edit, and deactivate tenant records via the React admin panel. High Priority.
FR-24	**Announcement Targeting**	Campus announcements shall support targeting by campus-wide, department, batch/session, or event types. City/district announcements shall support general public, emergency, event, and official notice types. Medium Priority.
FR-25	**Session Management**	Administrators shall be able to create and manage academic session records for campus tenants. Closed sessions shall block new signups from that batch. Medium Priority.
FR-26	**Verification Whitelist**	Administrators shall be able to maintain a dev_whitelist collection of emails that bypass email verification. Medium Priority.

3.4 Non-Functional Requirements

Security

- All communication between the mobile application, admin panel, and Firebase backend shall use HTTPS/TLS encryption protocols.

- User passwords shall be managed exclusively by Firebase Authentication; plaintext passwords shall never be stored.
- Firestore security rules shall enforce role-based data access, ensuring users cannot read or write data outside their permitted scope.
- User sessions shall automatically expire after a configurable inactivity period to prevent unauthorised access.
- OTP codes shall expire within five minutes of generation and be invalidated upon single use.

Performance

- All Firestore database queries shall complete within 2 seconds under standard network conditions (minimum 4G connectivity).
- The Home Dashboard and item listing screens shall load within 3 seconds following successful authentication.
- MobileNetV3 image embedding generation shall complete within 1.5 seconds on devices meeting minimum hardware specifications (2GB RAM, Android 8.0+).
- AI similarity computation shall be performed asynchronously to prevent UI thread blocking.

Reliability

- The system shall maintain 99% availability during academic operational hours by leveraging Firebase's globally distributed cloud infrastructure.
- The mobile application shall implement automatic retry logic for failed network requests (maximum 3 retries with exponential backoff).
- All Firestore writes shall be performed with transaction guarantees to prevent partial data corruption.

Usability

- The mobile application shall adhere to Material Design 3 guidelines to ensure interface consistency and familiarity.
- Primary tasks (reporting an item, submitting an incident) shall be completable within three navigation steps from the Home Dashboard.

- All interactive elements shall meet WCAG 2.1 minimum touch target size requirements (44×44 density-independent pixels).

Scalability

- The system architecture shall support a minimum of 1,000 concurrent authenticated users without performance degradation.
- Firestore composite indexes on (tenant_id ASC, createdAt DESC) shall maintain query performance as document counts scale to 100,000+ records.
- The multi-tenant shared database model shall support the addition of new tenant institutions without schema changes.
- The React admin panel's TenantSwitcher component shall allow super administrators to manage any number of tenant institutions from a single dashboard session.

CHAPTER 4

DESIGN AND ARCHITECTURE

4 Design and Architecture

4.1 System Architecture

CampusEase follows a three-tier client-server architecture with a clearly delineated presentation layer, business logic layer, and data layer. The system components are described below:

Flutter Mobile Application (Presentation and Client Logic Layer)

The Flutter mobile application serves as the primary interface for students and staff. Built in Dart using the Flutter SDK, it provides platform-adaptive UI components targeting Android with iOS compatibility. The application manages local state using Flutter's Provider pattern and implements offline caching via Firestore's native persistence layer [16]. On-device AI inference using TensorFlow Lite is invoked within the application for image embedding generation, keeping image data local and minimising network dependency for AI operations.

React Admin Dashboard (Administrative Presentation Layer)

The administrative web dashboard is developed using React.js and provides administrators with comprehensive operational oversight. It communicates directly with Firestore through the Firebase JavaScript SDK, rendering real-time data updates without page refresh. The dashboard implements role-based rendering, ensuring that admin-only actions (user management, claim approval) are inaccessible to lower-privilege authenticated sessions.

Firebase Backend (Business Logic and Data Layer)

Firebase provides the complete backend infrastructure. Firebase Authentication manages identity verification, session management, and role-based access. A secondary Firebase app instance (secondaryAuth) is used exclusively for admin-initiated staff account creation, preventing the admin's own session from being replaced by the newly created user's session. Cloud Firestore stores all structured data in a multi-tenant model: every document in posts, announcements, incidents, and users carries a tenant_id field, and all queries are scoped to the authenticated user's tenant. The system does not use Firebase Cloud Functions. Instead, all notification dispatch and push relay are handled by the Flutter NotificationService running on authenticated client devices, and all AI similarity computation is performed on-device within the Flutter application using TensorFlow Lite.

EmailJS

EmailJS is integrated in the React admin panel to send email notifications to staff applicants on approval or rejection. Because the admin panel is a purely client-side React application with no dedicated backend server, EmailJS provides a browser-callable email API using pre-configured templates, dispatching approval or rejection emails to the affected staff member's registered email address.

OneSignal Notification Service

OneSignal is integrated for push notification delivery [17], providing enhanced delivery reliability beyond the constraints of the Firebase free tier. Rather than invoking OneSignal from a backend server, the system uses a Firestore-queue pattern: when the React admin panel triggers a notification event, it writes a document to the pendingPushNotifications collection with status: pending. The Flutter NotificationService's startPushQueueListener() watches this collection and, upon detecting a pending document, calls the OneSignal REST API directly from the Flutter app, then marks the document as sent. This architecture avoids CORS restrictions (browser-to-OneSignal REST calls are blocked) while keeping the React panel free of any OneSignal dependency.

MobileNetV3 AI Module

The AI similarity module operates entirely on-device within the Flutter application using TensorFlow Lite. Upon item image submission, the MobileNetV3 model generates a 1280-dimensional feature embedding, which is stored in Firestore. The Flutter PostsProvider then retrieves the embeddings of all opposite-type items (lost vs. found) from Firestore and computes cosine similarity on-device within the application. No Cloud Functions or server-side inference are involved; all AI computation remains on the client device.

4.1.1 System Architecture Diagram

The CampusEase system **Figure 4.1:System Architecture Diagram** is built on a decentralized three-tier architecture that strategically offloads computational tasks to client-side environments to maintain a serverless, cost-effective infrastructure. The Frontend Layer comprises a Flutter mobile application for students and a React-based admin dashboard, both of which interact directly with the Firebase Backend Layer for authentication and real-time data persistence. A distinctive feature of this architecture is its "client-as-engine" approach: rather than utilizing server-side Cloud Functions, the system performs MobileNetV3 AI inference and cosine similarity matching locally on the mobile device, while also managing push notification dispatch via a Firestore-queue pattern where the

Flutter app acts as the relay for the OneSignal REST API. This design ensures high performance and data privacy by keeping heavy processing local, while Cloudinary and EmailJS provide specialized external support for image hosting and administrative communication respectively.

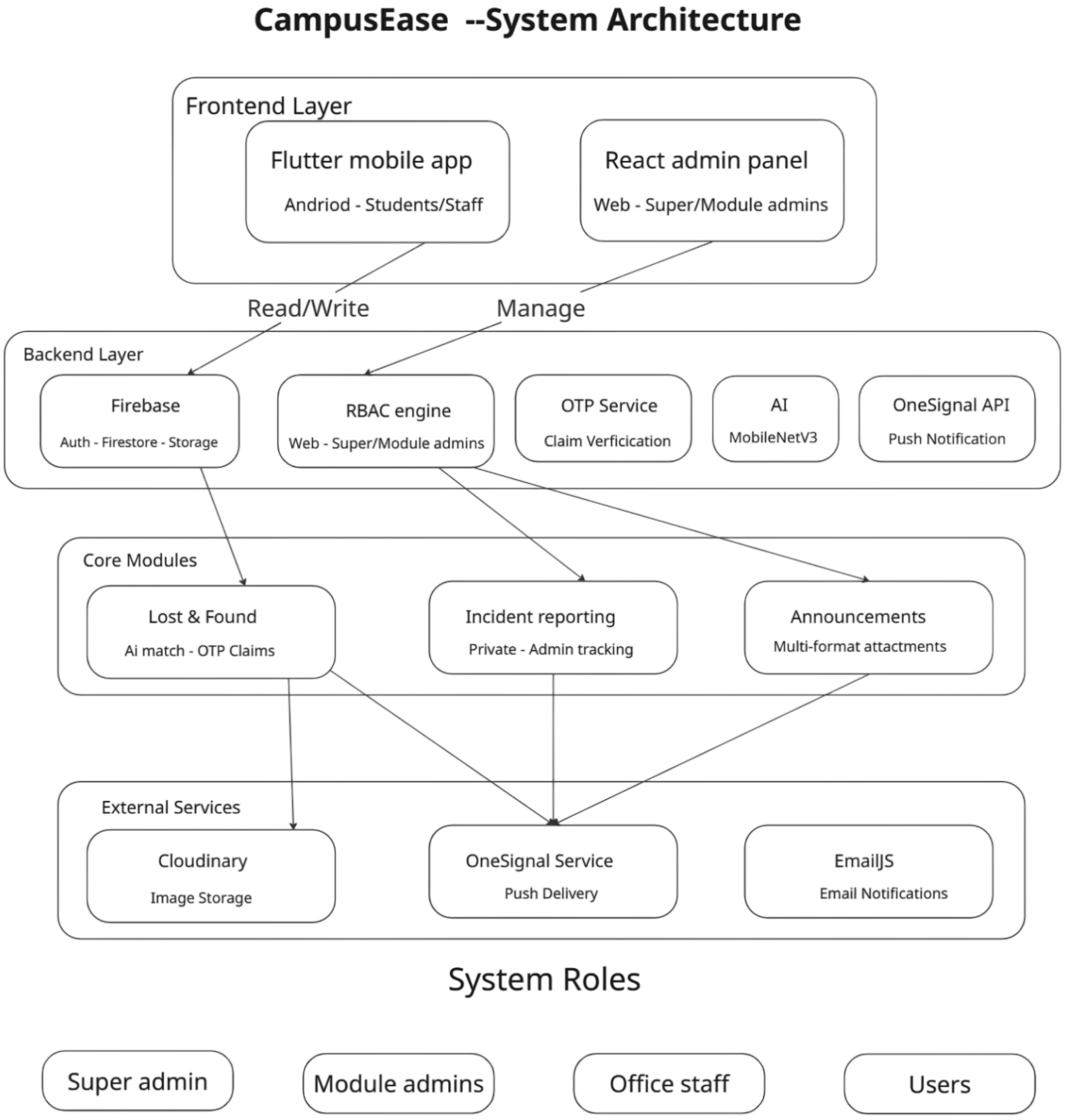

Figure 4.1:System Architecture Diagram

4.1.2 Simplified Four-Role System

CampusEase implements a deliberately minimal role hierarchy comprising four functional roles: student, staff, office, and public. Each role maps to a distinct set of application capabilities and UI navigation paths. Students are authenticated university members registered via institutional email with roll number and session verification. Staff are campus employees whose accounts are created by administrators and who have access to operational features equivalent to students, without roll-number constraints. Office accounts represent operational campus staff, such as security and administration personnel, who can create Lost and Found posts on behalf of others, with posts flagged postedByOffice=true in Firestore. Public accounts serve city and district tenant users who register freely without institutional email restrictions.

This four-role structure is intentionally simpler than a six-role or eight-role model: the pending and blocked states that might appear as separate roles in other systems are instead handled by the flag-based access control layer described in the following subsection. This design decision reduces the number of role-specific conditional branches required throughout the codebase and eliminates the risk of role explosion as the system scales to additional tenant types.

4.1.3 Flag-Based Access Control

Three boolean flags **Table 4.1** are stored on every user document in Firestore to represent access states that may change dynamically across the user lifecycle without altering the user's functional role. These flags are evaluated at authentication time as part of the three-layer login system and may also be checked at the point of feature access for additional security.

The justification for this design is rooted in the principle of role minimisation: system states such as pending approval, email unverified, and account blocked are orthogonal to functional role and should not require separate role types. A pending student and an active student have identical functional permissions upon approval; the distinction is purely a lifecycle state, not a capability difference. Modelling lifecycle states as flags rather than roles reduces role proliferation, simplifies Firestore security rule expressions, and allows administrators to modify user access states without reassigning roles.

Table 4.1:Authentication Flag Definitions

Flag	True Meaning	False Meaning	Effect on Login
active	Account is in good standing	Account is blocked or deactivated	active=false blocks login at Layer 2 regardless of all other flags
verified	Student has completed OTP email verification	OTP verification is pending	verified=false (students only) redirects to OTP verification screen
approved	Account has been approved by administrator	Account is pending admin review	approved=false redirects to pending approval screen; user cannot access main app

4.1.4 Three-Layer Authentication System

The **CampusEase** login validation system is structured as three sequential layers, each with a distinct security responsibility. This layered architecture separates identity verification, access state evaluation, and capability routing into independent concerns, improving maintainability and aligning with the defence-in-depth principles of NIST SP 800-53.

Layer 1 - Identity Verification

Upon credential submission, the system first verifies that a corresponding user document exists in the Firestore users collection. If no document is found for the authenticated Firebase UID, the session is immediately terminated and the user is informed that no account exists for the provided credentials. This gate prevents authenticated Firebase sessions from accessing the application without a corresponding application-level user record, closing the gap that exists when a Firebase account is created but the Firestore document write fails.

Layer 2 - Access State Verification

If a user document is found, the system evaluates the three boolean flags in sequence. First, if active=false, the session is terminated and the user is presented with an account blocked message. This handles both administrative deactivation and graduated student blocking. Second, if the user's role is student and verified=false, and the user's email is not present in the dev_whitelist collection, the user is redirected to the OTP verification screen rather than the main application. Third, if approved=false, the user is redirected to a pending approval screen informing them that their account

is under administrative review. No main application features are accessible until all three flags pass evaluation.

Layer 3 - Role-Based Routing

Once identity and access state are confirmed, the system reads the user's role field and routes them to the appropriate application shell. Students and staff are routed to the campus shell with access to all three modules. Office users are routed to the office shell with post-creation and claim management capabilities. Public users are routed to the city or district shell. This routing layer is entirely decoupled from the authentication gates, ensuring that capability assignment is a clean function of role and that future role additions require changes only to this routing layer.

4.2 Data Representation

The database is implemented in Firebase Cloud Firestore using a NoSQL document-collection model. The following table **Table 4.2:Firestore Data Model** describes the primary collections and their key fields:

Table 4.2:Firestore Data Model

Collection	Key Fields	Description
users	uid, email, fullName, role (student|staff|office|public), active (boolean), verified (boolean), approved (boolean), tenant_id, session (nullable), department (nullable), roll_number (nullable), deviceToken (OneSignal Player ID)	Authenticated users with functional role (student, staff, office, public). Access states managed via three boolean flags: active, verified, approved. Admin dashboard users are stored in the separate admins collection.
posts/{postId}	postId, userId, title, type (Lost|Found), status (active|under_claim|resolved), imageUrl, embedding[1280], tenant_id, postedByOffice, primaryClaimId, ownershipQuestions[]	Lost or Found item reports. Embedding stored for AI matching. postedByOffice flag

Collection	Key Fields	Description
		enforced by Firestore security rules.
posts/{id}/claims	claimId, userId, name, contact, action (claim|found), ownershipAnswers{}, imageUrl, status (full state machine), meetup{proposedTime, location}, otp (at otp_pending only), timestamps{}	Subcollection under each post. Full claim state machine data. OTP written only at otp_pending stage.
user_claims_index	user_claims_index/{uid}/claims/{claimId}: postId, postTitle, action, status, timestamp	Lightweight per-user index enabling My Claims screen to load without scanning the full posts collection.
match_suggestions	matchId, lostPostId, foundPostId, similarityScore, createdAt, reviewedByAdmin	AI-generated similarity matches linking lost and found post pairs. Written by Flutter PostsProvider after on-device cosine similarity computation.
disputes	disputeId, postId, claimId, raisedBy, reason, status (open|under_review|resolved)	Top-level dispute records created when either party raises a dispute or reports a no-show. Reviewed by admin panel.
audit_log	entryId, postId, claimId, userId, action, details, timestamp	Immutable audit trail of every claim state transition. Actions: claimSubmitted, claimAccepted,

Collection	Key Fields	Description
		claimRejected, claimCancelled, handoverComplete.
incident_reports	incidentId, reporterId (nullable for anonymity), category, description, location, imageUrl, status, assignedTo, timestamps{}	Structured campus incident reports with category-based routing and status fields.
announcements	announcementId, title, body, imageUrl, createdBy, targetRole, type, tenant_id, scheduledAt, createdAt	Campus-wide announcements created by administrators. Supports role targeting and scheduling.
notifications	notifId, userId, title, body, read, relatedModule, relatedDocId, isOwnerView, createdAt	In-app notification records across all modules. isOwnerView flag routes notification tap to correct screen.
tenants	id, name, type (campus|city|district), allowedDomain, studentDomain (campus only, absent for city/district), isActive	Master registry of all tenant institutions. studentDomain omitted using deleteField() for non-campus types.
sessions	id, name (e.g. FA22), label, tenantId, semester, isActive, createdAt	Academic batch/session records. Closed sessions (isActive: false) block new signups from that batch.
pendingPushNotifications	id, title, body, type, target (uid or ALL), status (pending|sent), createdAt	Notification queue written by React admin

Collection	Key Fields	Description
		panel. Flutter NotificationService relays each document to OneSignal and marks it sent.
dev_whitelist	id, email, note, createdAt, createdBy	Emails that bypass email verification. Loaded by Flutter app at startup via DevConfig.loadWhitelist().
admins	uid, name, email, role (super_admin|lostfound_admin|announcement_admin), tenant_id (non-super only), createdAt	Admin panel user records. Super admins have full cross-tenant access.

4.2.1 Entity-Relationship Diagram

The logical data model of CampusEase is Figure 4.2 structured around a multi-tenant architecture where the tenants collection serves as the root of the hierarchy, enforcing data isolation through a tenant_id foreign key present across all primary entities. User management is split between the users collection for mobile application stakeholders governed by role-based access and lifecycle flags and a separate admins collection for dashboard management. At the core of the functional modules, the posts collection manages lost and found items using on-device generated embeddings for AI matching, which are then linked via the ai_matches collection to facilitate recovery. Each post can host a claims subcollection that tracks the peer-to-peer state machine, including OTP verification and meetup scheduling. System integrity is maintained through the tracking of incidents and announcements, which manage structured reporting and targeted broadcasts respectively. These are all unified by a central notifications collection that integrates with the user base for real-time engagement.

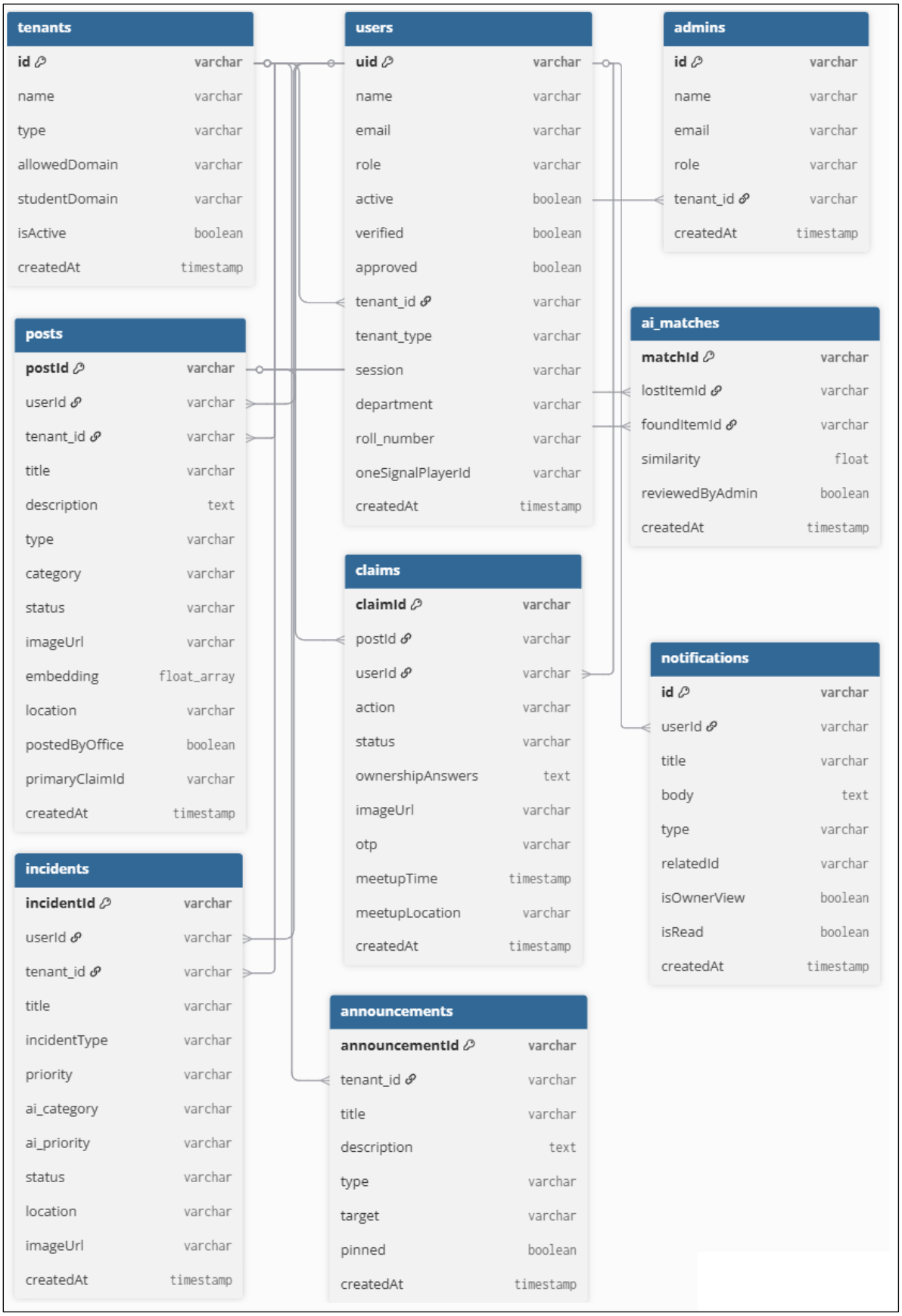

Figure 4.2:ER Diagram

4.2.2 Firestore Index Optimisation

The flag-based access control model requires composite Firestore indexes to support efficient administrative queries against multi-field filter combinations. The following composite indexes are defined in the project's Firestore index configuration:

Table 4.3:Firestore Composite Indexes for System Queries

Index (Collection + Fields)	**Purpose**
users: (tenant_id ASC, role ASC, approved ASC)	Admin panel query: pending approval staff filtered by tenant and role
users: (tenant_id ASC, role ASC, active ASC)	Admin panel query: blocked users by tenant
users: (tenant_id ASC, verified ASC)	Admin panel query: unverified users awaiting OTP completion
posts: (tenant_id ASC, createdAt DESC)	Main feed query: tenant-scoped items in reverse chronological order
announcements: (tenant_id ASC, createdAt DESC)	Announcements feed with tenant isolation
incidents: (tenant_id ASC, createdAt DESC)	Incidents list with tenant isolation
sessions: (tenantId ASC, seq ASC)	Ordered session list for campus tenant signup dropdowns

4.3 Process Flow / Representation

4.3.1 Core Module Workflows

4.3.1.1 Lost Item Reporting and AI Matching Workflow

An authenticated user submits the lost item form with image. The image is processed locally by MobileNetV3 to generate a 1280-dimensional embedding, which is stored in Firestore alongside the item record. The Flutter PostsProvider then retrieves the embeddings of all items of the opposite type from Firestore and computes cosine similarity on-device. All matches above the 0.70 threshold are written to the match_suggestions collection, and a push notification is dispatched to the item owner via the OneSignal queue pattern.

4.3.1.2 Claim Verification Workflow

The claim workflow is a peer-to-peer, six-stage state machine that operates entirely between the

claimant and the post holder, with no admin involvement unless a dispute is raised. Stage 1 (pending): the claimant submits ClaimOrFoundFormPage; ClaimProvider executes a Firestore batch write. Stage 2 (accepted/rejected): the holder reviews all submitted details, ownership answers, and proof image in HolderClaimManageScreen and accepts or rejects. Stage 3 (meetup_proposed/meetup_confirmed): the holder proposes a meetup; the claimant accepts or counter-proposes until mutual confirmation. Stage 4 (otp_pending → handover_pending → completed): the holder generates the OTP on their device; the claimant enters it on theirs; the holder confirms physical handover and the record advances to completed. Disputes are available to either party at any stage.

4.3.1.3 Incident Reporting Workflow

A user submits an incident report with category, description, and location. The system routes the report to the appropriate authority based on incident category. The assigned administrator or staff member receives a notification, reviews the report, and updates the status. Status changes trigger push notifications to the reporting user.

4.3.1.4 Announcement Broadcasting Workflow

An administrator creates an announcement in the React admin dashboard. Upon publication, the announcement document is created in Firestore and the React panel simultaneously writes a pending document to the pendingPushNotifications collection. The Flutter NotificationService, running on an authenticated device, detects the pending document via its Firestore queue listener, calls the OneSignal REST API directly, and marks the document as sent. The announcement also appears in the in-app notification feed for users who access the application offline and subsequently come online.

4.3.2 Authentication and Governance

4.3.2.1 Signup Flow with Semester Validation

The campus tenant signup flow implements semester-based governance logic that enforces real-world university enrollment constraints. When a student selects their academic session during registration, the system applies the following validation rules based on the semester number associated with the selected session:

- **Semesters 1 through 8**

The student is assigned approved=true on account creation, granting immediate access to the campus

application shell upon successful OTP verification. This reflects the standard active enrollment period for undergraduate programmes.

- **Semesters 9 and 10**

The student is assigned approved=false on account creation. After OTP verification, they are redirected to a pending approval screen and cannot access the main application until an administrator explicitly approves their account. This models the extended enrollment status common in Pakistani university programmes, where students beyond the standard eight-semester duration require administrative confirmation of continued enrollment.

- **Semester 11 and above**

Signup is blocked with a validation error at the form level. No account is created for students claiming a semester number beyond the maximum supported value, preventing data entry errors and enforcing graduation boundaries.

On account creation for all campus students, the initial flag state is set as follows: active=true (account is in good standing at signup), verified=false (OTP not yet completed), and approved=(true for semesters 1 to 8, false for semesters 9 to 10). This flag initialisation is performed atomically as part of the Firestore user document write during signup, ensuring no partial state is observable.

4.3.2.2 OTP Verification Flow

Email OTP verification in **CampusEase** is implemented using EmailJS rather than Firebase's built-in email verification mechanism. Firebase's email verification link mechanism redirects users to a browser-based URL that disrupts the native Flutter application flow, requiring custom deep linking configuration that adds complexity without functional benefit. EmailJS enables delivery through institutional SMTP credentials configured in the EmailJS dashboard, allowing verification emails to originate from a university-branded sender address.

The OTP flow proceeds as follows: upon account creation, a six-digit numeric code is generated and stored in the user's Firestore document with an otpExpiresAt timestamp set five minutes in the future. An EmailJS API call delivers the OTP to the user's registered email address. The user enters the OTP in the Flutter application. Upon correct entry, the system validates the code against the stored value and checks the expiry timestamp. If both conditions are met, verified is set to true and the otp and otpExpiresAt fields are deleted. The user is then routed according to their approved flag.

4.3.2.3 Admin Approval Workflow

The admin approval workflow handles two categories of users requiring administrative review: campus students in semesters 9 and 10 (Extended Students), and staff applicants registered via self-registration. Both categories are surfaced in the React admin dashboard with pending status indicators.

For Extended Students, the admin panel presents a dedicated Extended Students page listing all users with role=student and approved=false within the active tenant. Administrators can approve, setting approved=true and dispatching an EmailJS notification, or reject, setting active=false and dispatching an EmailJS rejection email with an optional reason. For staff applicants, the Staff Accounts page highlights all users with role=staff and approved=false with a pending badge. Both workflows use the fetch-before-update pattern to ensure that email address and name are read from Firestore at the time of the action, preventing stale local state from being used in email dispatch.

4.4 User Process Flow

The **CampusEase** platform implements a unified user process flow that encompasses registration, authentication, access control, and module navigation. The flow incorporates three critical validation stages:

(1) **user identity verification** through email and password credentials,

(2) **flag-based access state evaluation** (active, verified, approved) that gates access to application features, and

(3) **role-based routing** that directs authenticated users to their appropriate application shell (campus shell for student/staff, office shell for office users, or city/district shell for public users).

For users accessing the Lost and Found module, the workflow details the complete journey from item discovery through claim submission, holder review, and meetup scheduling, culminating in an OTP-verified handover. This process represents the core operational logic of the platform, demonstrating the multi-stage peer-to-peer verification mechanism that ensures secure item transfer. The following Process flow diagram **Figure4.3:User Process Flow Diagram** illustrates the complete user journey from application launch through successful module access, including decision gates for authentication, flag-based access verification, and role-based routing.

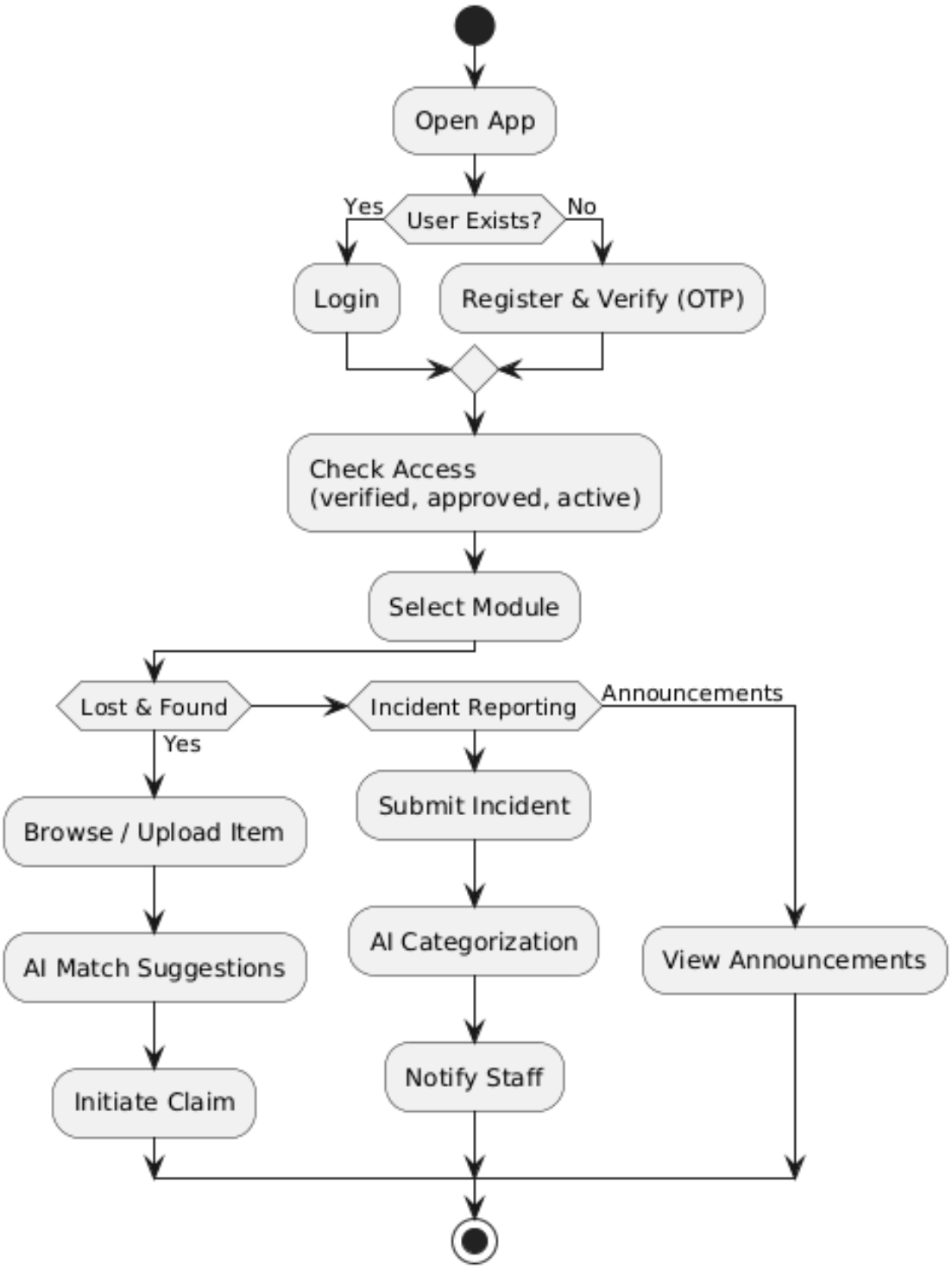

Figure4.3:User Process Flow Diagram

4.5 Admin Process Flow

The **CampusEase** administrative dashboard, developed in React, facilitates governance through a Role-Based Access Control (RBAC) hierarchy. This structure ensures that system capabilities are strictly partitioned based on the administrator's scope:

- **Super Admin**

Manages cross-tenant operations, including campus creation, global system analytics, and the assignment of campus-level administrators.

- **Campus Admin**

Handles institution-specific governance, such as student/staff flag management (active, approved), user approval workflows, and broadcasting campus-wide announcements.

- **Module Admin**

Operates with high granularity, focusing on monitoring logs and resolving disputes within specifically assigned modules like Lost & Found or Incident Reporting.

Beyond simple authentication, the workflow emphasizes proactive system monitoring. Administrators track AI-driven actions such as automated matching and triage and intervene during disputes or detected abuse. The "Case Review" panel allows for an evidence-based audit of images and history before an action (blocking, flagging, or warning) is finalized.

Every administrative decision, from a status update to an announcement broadcast, is logged to an immutable audit trail and simultaneously triggers a real-time push notification via the OneSignal queue pattern, ensuring immediate synchronization between the web dashboard and the mobile client.

The following process flow diagram **Figure 4.4:Admin Process Flow Diagram** illustrates the lifecycle of administrative oversight:

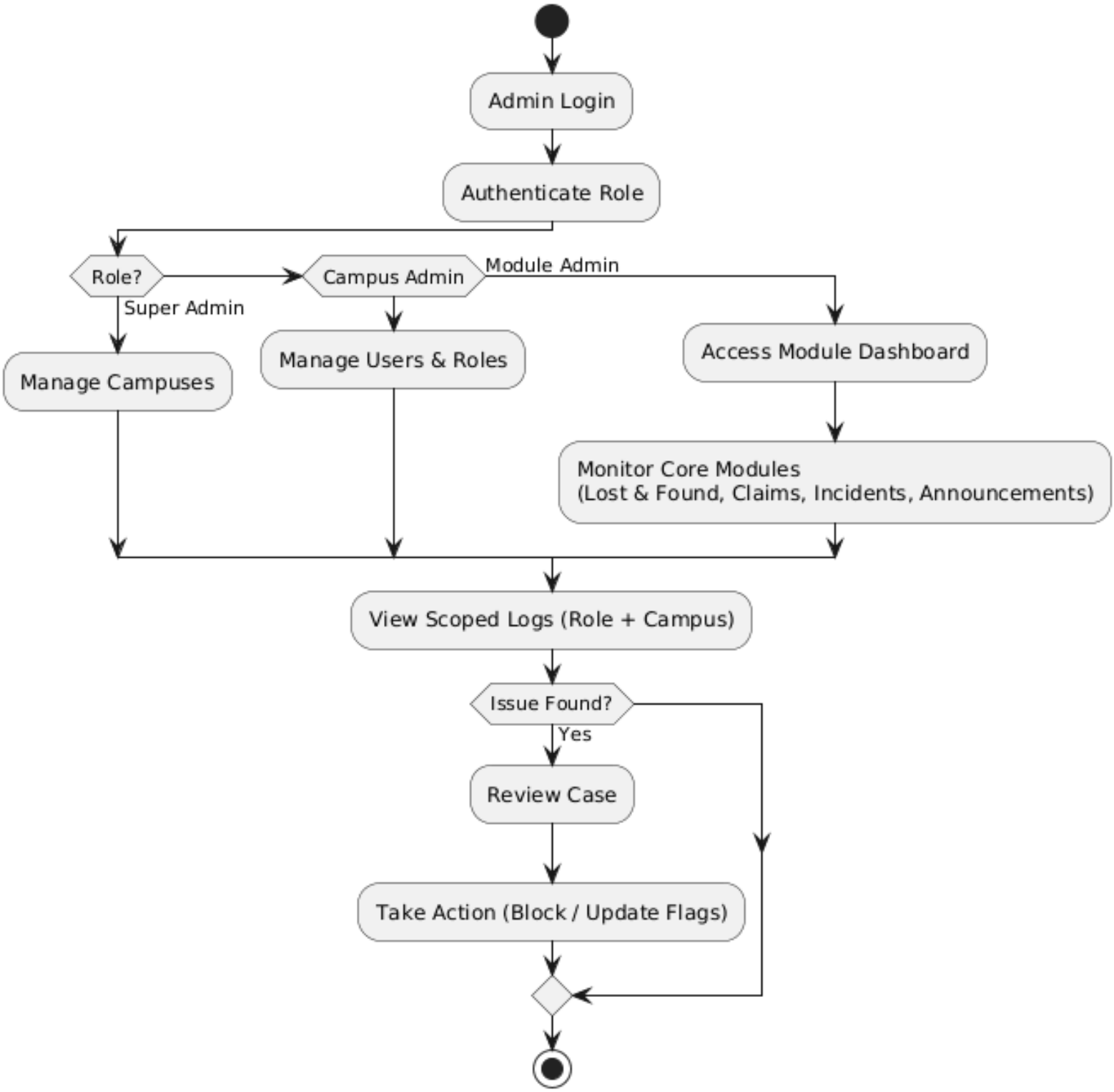

Figure 4.4:Admin Process Flow Diagram

4.6 Design Models

4.6.1 Sequence Diagram Report Item Submission

The following sequence diagram **Figure 4.5:Report item sequence** details the synchronous message exchange and background processing required to submit a lost or found item while initiating the AI matching engine.

The process begins with the Flutter Mobile App performing a security handshake with Firestore to verify the user's authentication and tenant identity. Upon form completion, the application executes a parallel data processing pipeline:

- **External Asset Management**

The selected image is uploaded to Cloudinary, which returns a secure HTTPS URL for persistent storage.

- **On-Device AI Inference**

Simultaneously, the image is passed to the local MobileNetV3 model. The image is preprocessed to a 224×224 pixel resolution before the model performs inference to generate a 1280-dimensional embedding vector.

- **Data Persistence**

The Flutter app writes the complete item document incorporating the tenant_id, AI embedding, Cloudinary URL, and metadata to Firestore.

- **On-Device Matching Engine**

Rather than utilizing a server, the app retrieves embeddings of all opposite-type items from the local cache/Firestore. It then computes a hybrid similarity score using a weighted formula: 70% Image Cosine Similarity and 30% TF-IDF Text Similarity.

- **Notification & Success**

Any candidate pairs meeting or exceeding the 0.70 threshold are committed to the match_suggestions collection. The OneSignal REST API is then invoked to queue a push notification. The workflow concludes by rendering the matched items directly to the user, sorted by their similarity percentage scores.

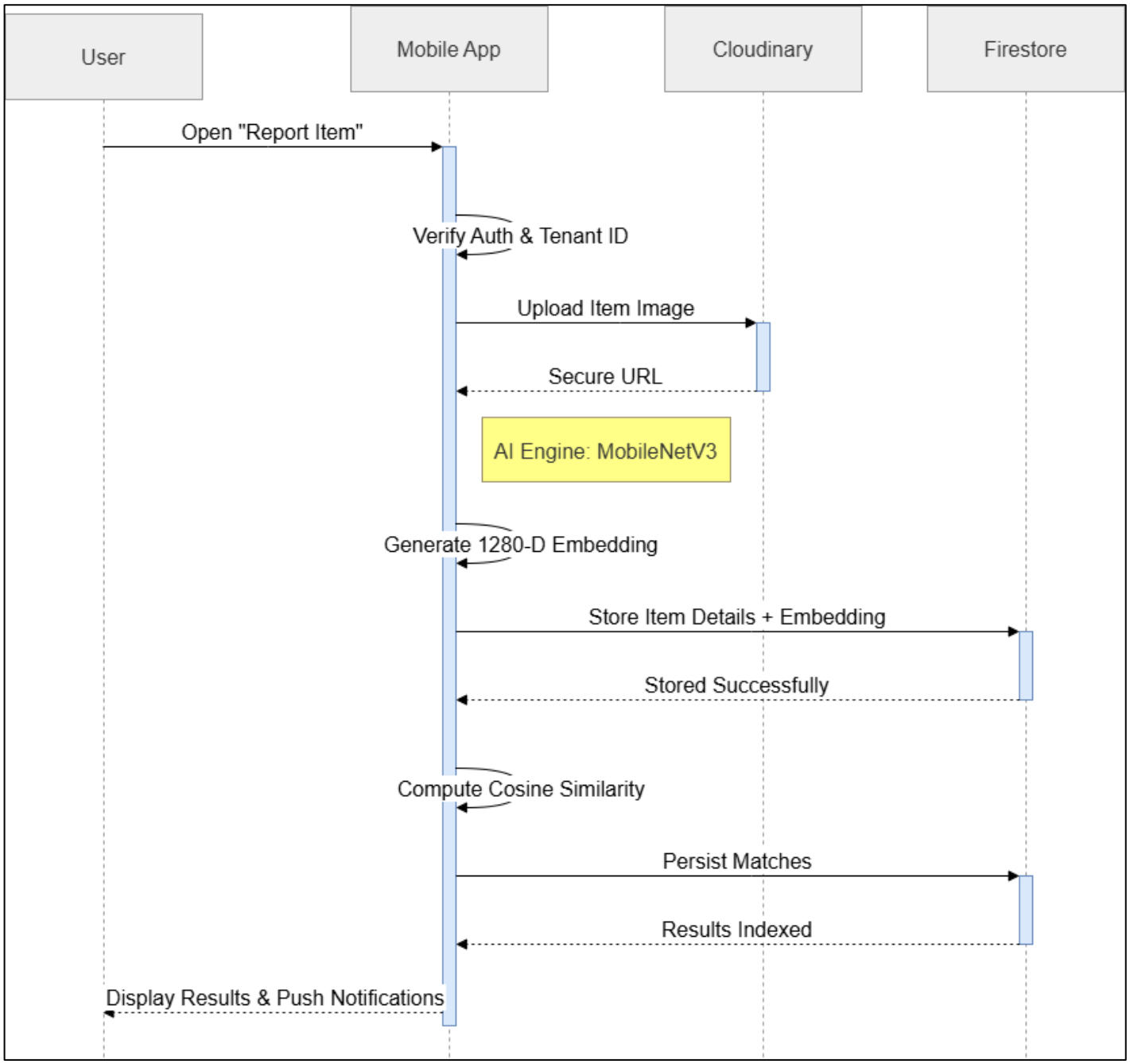

Figure 4.5:Report item sequence

4.6.2 Sequence Diagram Claim State Machine

The sequence diagram **Figure 4.6:Claim Sequence Diagram** captures the full six-stage state machine that governs the decentralized handover process. This workflow operates as a peer-to-peer interaction between the Claimant and the Holder, coordinated through atomic database operations to ensure data consistency.

The process is initiated when the Claimant submits the ClaimOrFoundFormPage. The ClaimProvider executes a Firestore batch write, which simultaneously creates the claim document, updates the user_claims_index, and generates an entry in the AuditService. Upon submission, the NotificationService dispatches a push notification to the Holder via the OneSignal queue.

The workflow then proceeds through several critical synchronization gates:

- **Review & Acceptance**

The Holder utilizes the HolderClaimManageScreen to review ownership answers and proof images. Accepting the claim triggers a second atomic batch update that modifies the claim status, post state, and user indices.

- **Meetup Negotiation**

The MeetupProvider facilitates a reciprocal exchange where the Holder and Claimant propose and counter-propose meeting times until a mutual meetup_confirmed state is reached.

- **OTP-Verified Handover:**

At the physical meetup, the Holder invokes OtpProvider.generateOtp(), which writes a six-digit code to Firestore with a five-minute TTL (Time-to-Live) and renders it exclusively on the Holder's device. The Claimant then enters this code via OtpProvider.verifyOtp(); upon successful validation, the state advances to handover_pending.

- **Completion & Auditing**

The process concludes when the Holder calls HandoverProvider.confirmHandover(). This finalizes the claim as completed, marks the item post as resolved, and triggers the AuditService to log the handoverComplete event.

Throughout this lifecycle, a Dispute branch remains available to both parties. If triggered, the flow diverges to escalate the case to the Admin Dashboard for manual evidence review and resolution, ensuring a fail-safe mechanism is present at every stage of the peer-to-peer exchange.

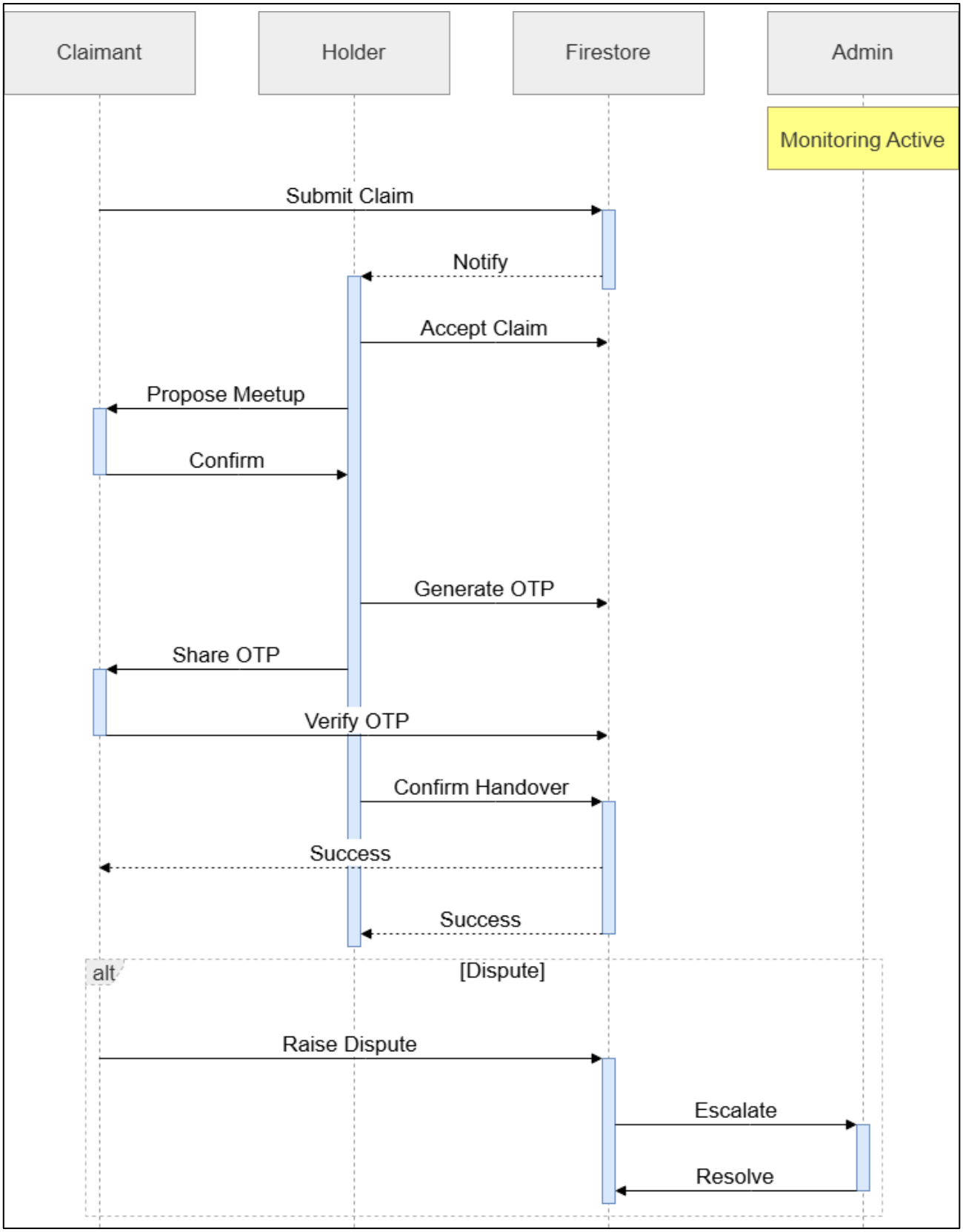

Figure 4.6:Claim Sequence Diagram

4.6.3 Sequence Diagram Incident Reporting

This sequence diagram illustrates the automated workflow and real-time data synchronization of the **CampusEase** Incident Reporting module. The process begins with a user submission on the **Mobile App**, followed by **Gemini AI** analysis for categorization and prioritization. Once the data is persisted to **Firestore**, it is synchronized with the **Web Admin Panel** for review. The **Admin** then updates the status, which is pushed back to the **User** via a real-time listener, completing the feedback loop.

- **AI-Powered Automated Triage**

By integrating the **Gemini API**, the system eliminates the bottleneck of manual sorting, instantly analyzing and flagging high-priority emergencies to ensure critical campus issues receive immediate administrative attention.

- **Real-Time Reactive Synchronization**

The architecture utilizes **Firestore** as a real-time bridge, enabling seamless data flow between the Mobile and Web platforms without the latency or overhead associated with traditional, request-response REST API polling.

- **End-to-End State Management**

The workflow demonstrates a robust state-transition model where every administrative action—from acknowledgment to resolution—directly triggers a user-side notification, maintaining full transparency throughout the incident lifecycle.

- **Multi-Tenant Data Integrity**

The diagram highlights a secure architectural efficiency where data isolation is maintained through Tenant ID verification, ensuring that incident reports are strictly routed and accessible only within their specific university context.

- **Scalable Intelligence Integration**

By performing semantic analysis before persistence, the system ensures that every record in the database is already enriched with AI insights, allowing for advanced filtering and rapid response times by campus authorities.

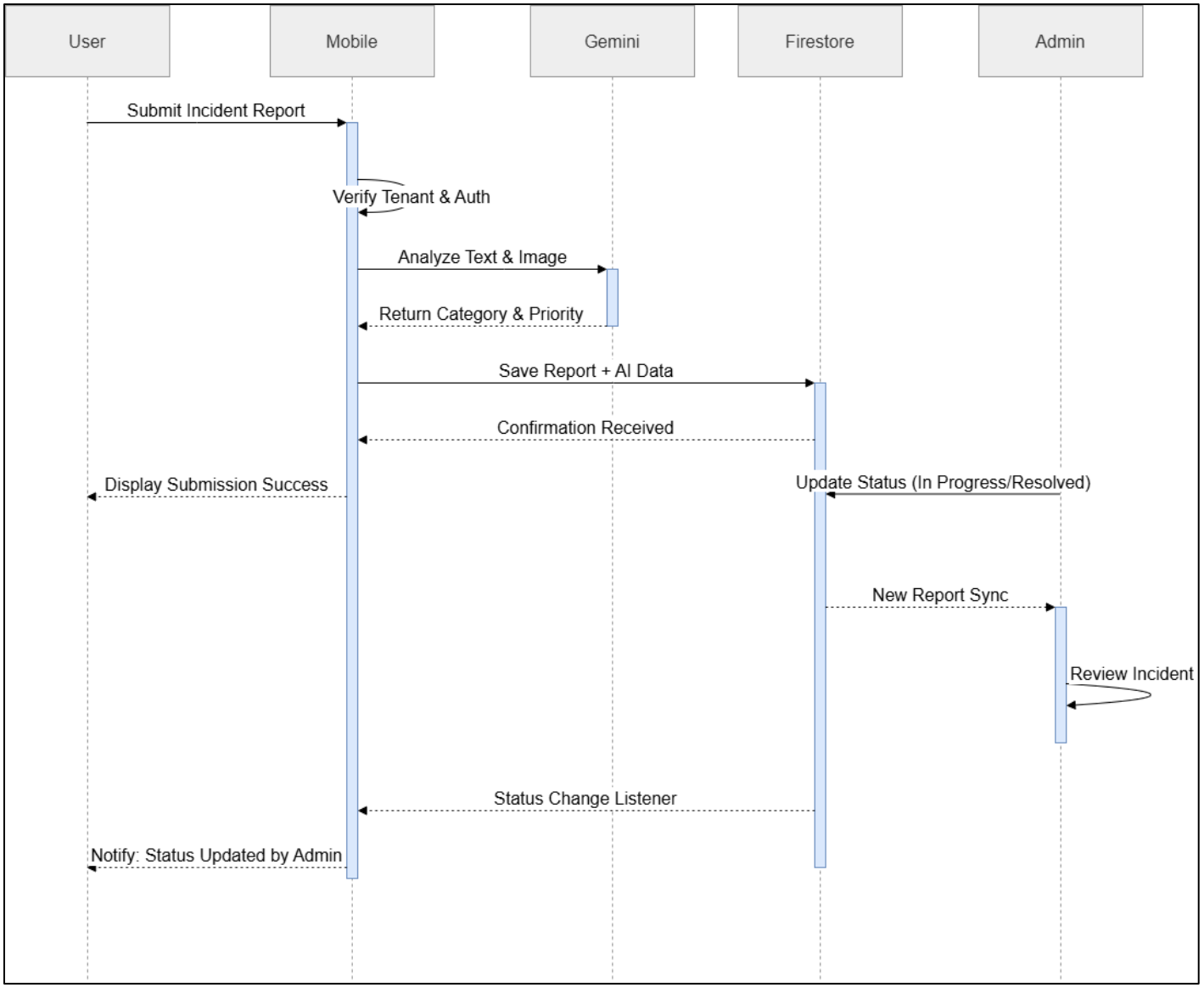

Figure 4.7:Incident Sequence Diagram

4.6.4 Component Diagram

The following component diagram **Figure 4.8:Component Diagram** illustrates the modular decomposition of the **CampusEase** system and the dependencies between its primary execution environments and external services.

The system is organized into four major logical containers:

- **Flutter Mobile Application**

This client-side component serves as the primary execution engine. It houses internal modules for Authentication, Lost & Found, Incidents, and Announcements. Critically, it includes a Notification

Service that manages push relay logic and a local TFLite AI Engine (MobileNetV3) for on-device embedding generation.

- **React Admin Panel**

A web-based management interface that encapsulates Staff Management, Announcements Manager, and Incidents Manager sub-components. It maintains administrative context and session whitelisting while communicating directly with the data layer via the Firebase JS SDK.

- **Firebase Backend**

Acts as the centralized persistence and identity layer. It consists of Firebase Auth for secure access, Firestore for multi-tenant document storage, and Cloud Storage for binary data. Following the serverless client-side logic pattern, no Cloud Functions are deployed within this component.

- **External Services**

The system integrates specialized third-party APIs to extend functionality, including OneSignal REST for push delivery, Google Gemini for AI-assisted triage, Cloudinary CDN for optimized image hosting, and EmailJS for administrative email dispatch.

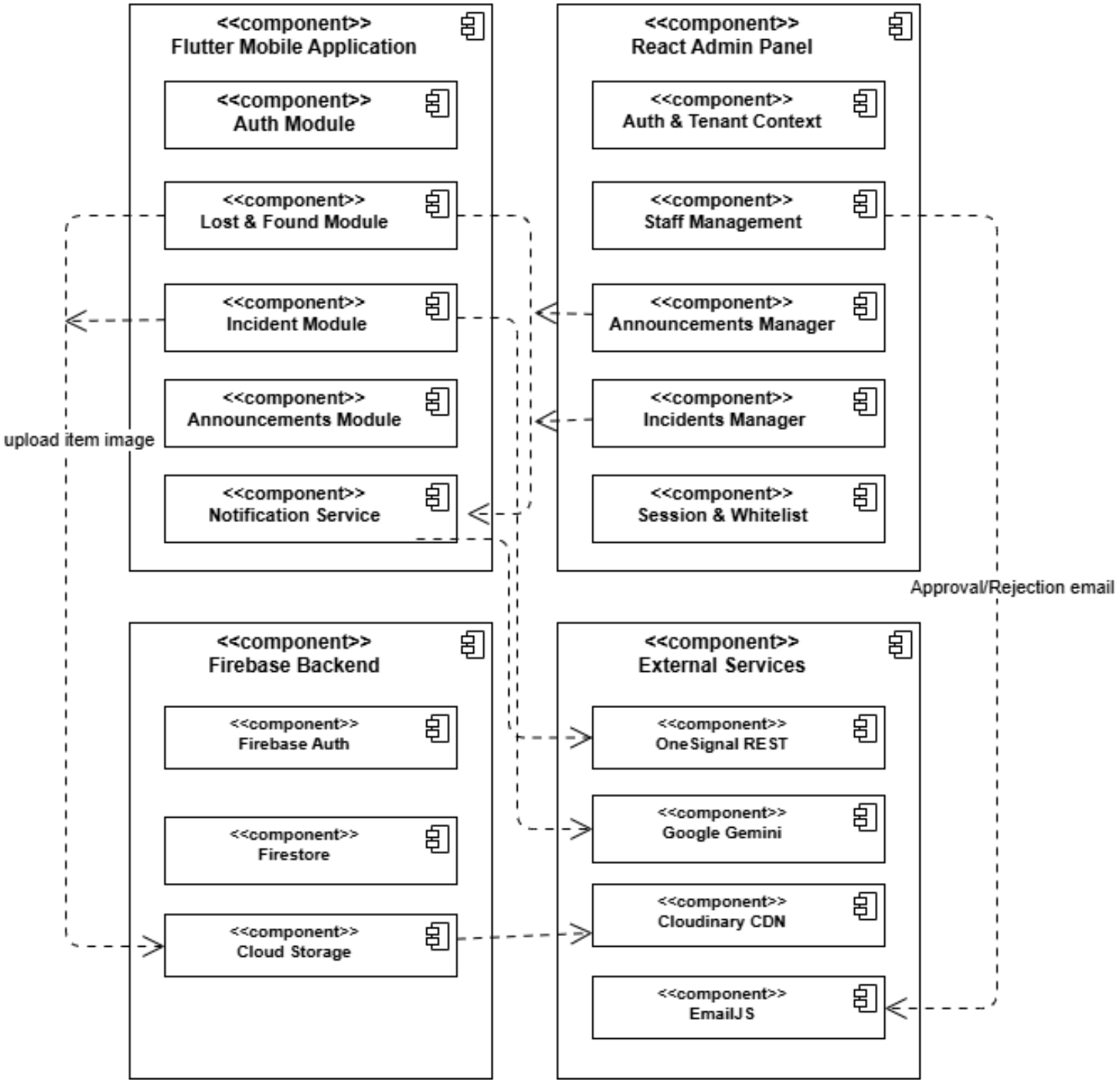

Figure 4.8:Component Diagram

4.6.5 Class Diagram

The class diagram **Figure 4.9:Class Diagram** illustrates the static structural model of the **CampusEase** platform, defining the essential entities and the logical relationships that govern multi-tenant data integrity.

The data model is centered around several core architectural pillars:

- **Identity & Governance:** The Global_Super_Admin manages the high-level Tenant entities, which in turn host User accounts. The User class is central to the system, containing role-based and tenant-specific attributes that dictate access to the application shells.
- **Operational Entities:** The Post class represents the primary data unit for the Lost & Found module, featuring a 1280-dimensional embedding array for AI matching. It maintains a one-to-many relationship with Claim and AI_Match objects.
- **Workflow Integrity:** The peer-to-peer verification logic is supported by the Claim and Claim_Audit_Log classes, ensuring every transition from discovery to handover is tracked.
- **Communication & Safety:** The Incident, Announcement, and Notification classes handle the flow of information and safety reporting, with associations back to the User to ensure notifications are received by the correct recipients.

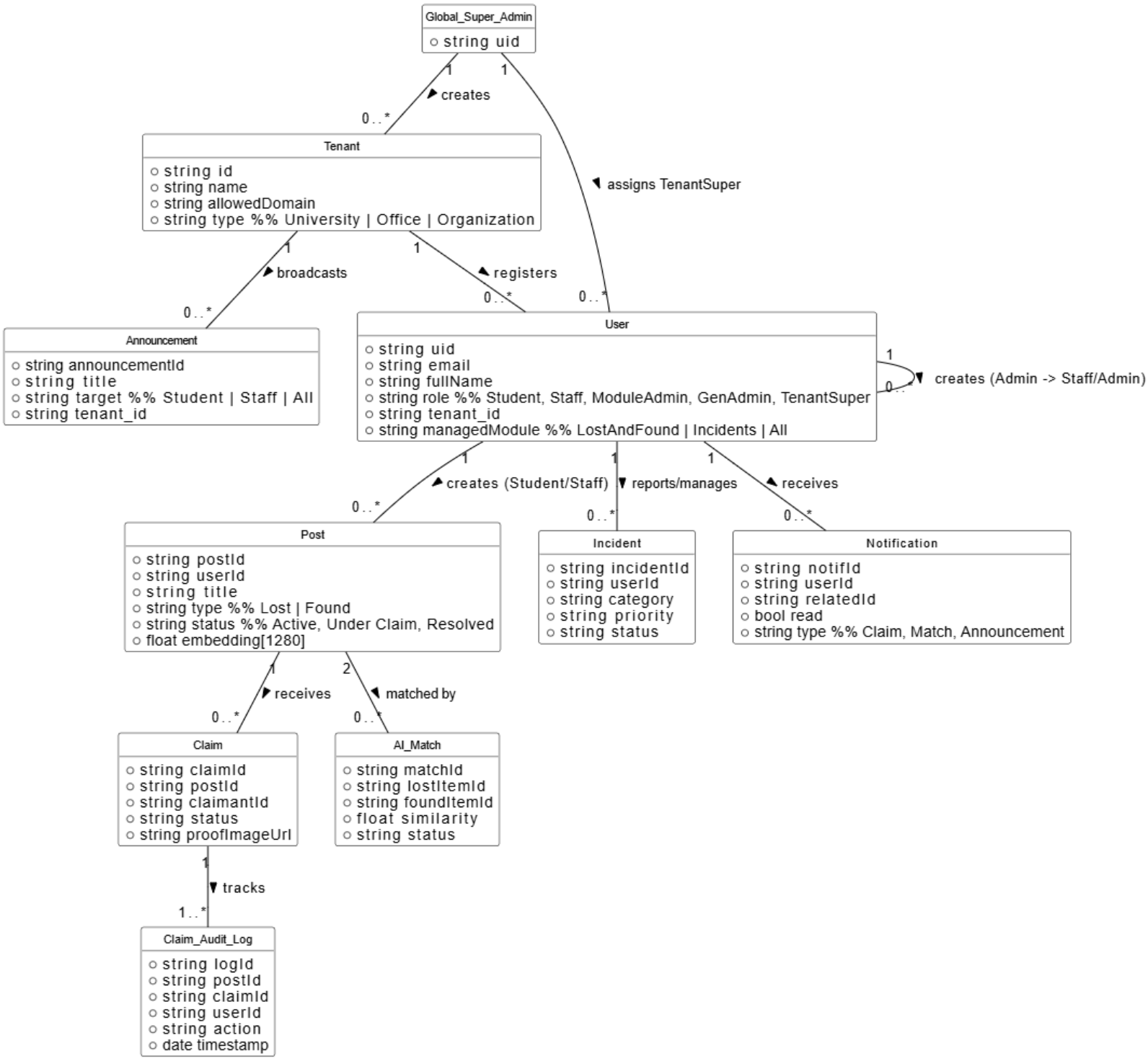

Figure 4.9:Class Diagram

CHAPTER 5
IMPLEMENTATION

5 Implementation

5.1 Algorithms

5.1.1 AI Image Similarity Algorithm

The AI image similarity algorithm implemented in **CampusEase** operates through three conceptual stages: feature extraction, embedding storage, and similarity computation.

Stage 1: Feature Extraction (On-Device)

When a user submits an item image, it is preprocessed to 224×224 pixels and normalised to the [0, 1] range as required by MobileNetV3. The TensorFlow Lite interpreter loads the pre-converted MobileNetV3 model file (.tflite) and performs forward inference on the preprocessed image. The output of the model's penultimate fully-connected layer is a 1280-dimensional floating-point vector the embedding which serves as a compact numerical representation of the image's visual features.

Stage 2: Embedding Storage

The generated embedding array is stored as a field within the corresponding Firestore document (lost or found items). The Flutter PostsProvider, running within the mobile application, then retrieves the stored embeddings of all items of the opposite type from Firestore to initiate the matching workflow. No server-side trigger or Cloud Function is involved.

Stage 3: Similarity Computation

Upon new item submission, the Flutter PostsProvider retrieves all embedding vectors of the opposing item type from Firestore. For each candidate pair (A, B), cosine similarity is computed using the formula:

$$\text{similarity}(A, B) = (A \cdot B) / (|A| \times |B|)$$

Where A · B is the dot product of the two embedding vectors, and |A| and |B| are their respective Euclidean norms. This formula yields a similarity score in the range [0, 1], where 1 indicates identical visual features and 0 indicates complete dissimilarity. All pairs achieving a score of 0.70 or greater are written to the match_suggestions collection with their respective scores.

Text-based similarity is computed in parallel using TF-IDF vector representations of item descriptions, with cosine similarity computed between description vectors. The final match score

presented to users represents a weighted combination of image similarity (70% weight) and text similarity (30% weight), providing a more robust match estimation than either signal alone.

5.1.2 Semester Determination Algorithm

The semester number associated with a student's academic session is determined dynamically from the Firestore sessions collection at signup time. The algorithm retrieves all session documents for the active tenant, ordered by the seq field in ascending order. When a student selects a session from the signup dropdown, the corresponding session document's semester field determines the student's current semester. The flag assignment logic applies the following rule: if semester is less than or equal to 8, set approved=true; if semester is 9 or 10, set approved=false; if semester is greater than 10, block signup with a validation error before any Firestore write is performed.

5.1.3 Flag-Based Access Control Algorithm

The three-layer authentication algorithm is implemented in the Flutter LoginViewModel and evaluated sequentially on every login attempt. The pseudocode for the complete evaluation chain is: call Firebase Authentication signInWithEmailAndPassword; if authentication fails, return AuthError; fetch userDoc from Firestore users collection where uid equals currentUser.uid; if userDoc does not exist, call signOut() and return AccountNotFoundError; if userDoc.active equals false, call signOut() and return AccountDeactivatedError; if userDoc.role equals student and userDoc.verified equals false and email is not in DevConfig.whitelist, navigate to OtpVerificationScreen and return; if userDoc.approved equals false, navigate to PendingApprovalScreen and return; route user to shell determined by userDoc.role and return AuthSuccess.

5.1.4 Safe Mode Backfill Algorithm

The Safe Mode backfill algorithm applies missing flag fields to existing user documents without overwriting any existing values. The pseudocode is: initialise lastDocument to null; loop until no more documents: query users collection where tenant_id equals tenantId, limit 400, starting after lastDocument; if zero documents returned, exit loop; initialise Firestore batch; for each document: compute missingFlags as the set of flag fields absent from the document; for each missing flag, determine the appropriate default value based on role and semester; if missingFlags is non-empty, add batch update for that document; set lastDocument to the last document in the result set; commit

batch; end loop; return a completion report with counts of documents processed, updated, and skipped.

5.2 External APIs and Integrations

5.2.1 Firebase Services

Firebase Authentication manages identity, session management, and account provisioning across both the Flutter mobile app and React admin dashboard via their respective native SDKs, without relying on Cloud Functions. Cloud Firestore acts as the primary database, utilizing real-time listeners for automatic UI updates and scoping all queries to the authenticated user's tenant_id to ensure strict multi-tenant isolation. To prevent the active administrator's session from being overwritten during staff account creation, a secondary Firebase app instance is initialized exclusively for admin-initiated provisioning. For extended services, Firebase Cloud Messaging manages the device token infrastructure alongside OneSignal for targeted push notifications, while Firebase Remote Config stores the Google Gemini API key to allow server-side rotation without application rebuilds. Finally, Firebase Cloud Storage provides supplementary file storage, complementing Cloudinary, which serves as the primary hosting solution for item and announcement images.

5.2.2 OneSignal

OneSignal is integrated via its REST API, called directly from the Flutter application's NotificationService class. When a notification event is triggered a match suggestion, claim status update, incident status change, or announcement the React admin panel writes a document to the Firestore pendingPushNotifications collection with status: pending. The Flutter NotificationService.startPushQueueListener() method monitors this collection in real time. Upon detecting a pending document, it constructs the OneSignal REST API payload, applies the correct targeting strategy (include_aliases for single users, tag filters for department or batch, included_segments for broadcast), sends the POST request, and marks the Firestore document as sent. This queue-based pattern avoids the CORS restriction that prevents browsers from calling the OneSignal REST API directly, while keeping the React admin panel entirely decoupled from OneSignal.

5.2.3 MobileNetV3

The MobileNetV3-Large model is converted to TensorFlow Lite format (.tflite) and bundled with the Flutter application as an asset. The tflite Flutter plugin loads the model into memory on application initialisation. Image preprocessing (resizing to 224×224 and normalisation) is performed using the image Dart package prior to model inference. The TFLite interpreter returns the 1280-dimensional embedding, which is subsequently serialised as a JSON array and stored in Firestore.

5.2.4 Cloudinary

Cloudinary serves as the primary image hosting solution. The Flutter application uses the Cloudinary upload API to POST image files, receiving a secure HTTPS URL in response. This URL is stored in the Firestore item document, enabling the admin panel and mobile client to display images without direct Firebase Storage access.

5.2.5 EmailJS

EmailJS provides browser-callable email delivery used across three workflows in the CampusEase ecosystem. In the React admin panel, EmailJS is invoked client-side from StaffContext.jsx using pre-configured templates to dispatch staff approval and rejection notifications, eliminating the need for a dedicated backend server for transactional email. In the Flutter mobile application, EmailJS is used to deliver the six-digit OTP to the user's registered email address during the account verification flow, replacing Firebase Authentication's default email verification mechanism to ensure reliable delivery through institutional SMTP configurations and to maintain a native in-app verification experience without browser redirection. The same EmailJS configuration is additionally used by the Vercel serverless function to deliver password reset links, ensuring consistent institutional branding and reliable delivery across all system-generated emails.

5.2.6 Google Gemini API

The Incident Reporting Module integrates the Google Gemini 2.5 Flash Lite API to provide AI-assisted triage of submitted incident reports. When a user submits an incident description, the application forwards the text to Gemini via a secure API call managed through Firebase Remote Config (which stores the API key server-side, enabling key rotation without requiring an application rebuild). Gemini analyses the description and returns a structured JSON response containing a suggested incident category (e.g., Security, Safety, Maintenance, Behavioural) and a recommended priority level (low, medium, or high). These AI-generated suggestions are stored as separate fields ai_category and ai_priority alongside the user-provided values, and are surfaced to the admin in the

React dashboard as advisory classifications. The system is designed with graceful degradation: if the Gemini API is unavailable or the request times out (default threshold: 10 seconds), the incident is stored with the user-supplied values only, and AI triage is marked as deferred. This integration significantly reduces the cognitive burden on campus administrators by pre-classifying incoming reports and highlighting high-priority incidents, improving response time and resource allocation efficiency.

5.2.7 CampusEase Web Platform

The CampusEase web platform, deployed on Vercel, functions as both the public-facing entry point and the backend routing layer for the ecosystem. Globally, it provides the official landing page, legal documentation (Privacy Policy, Terms and Conditions), and an APK download endpoint managed via an administrative Firestore toggle to enable controlled releases without redeploying code. As a routing layer, the platform hosts deep-link redirection endpoints that render guest previews for devices without the application, alongside an out-of-band password reset handler that bridges users back to the Flutter app upon successful updates. To bypass institutional email filtering restrictions that block default Firebase communications, this password reset flow is implemented as a stateless, rate-limited serverless function; it leverages the Firebase Admin SDK and EmailJS for reliable delivery, utilizing the unified Firebase project and EmailJS configuration shared across the mobile application and React admin panel.

5.3 User Interface

The following sections describe the key user interface screens of the CampusEase mobile application and admin dashboard.

5.3.1 Mobile Application Interface

5.3.1.1 Registration and Login Screen

The Registration and Login screen, shown in Figure 5.1, serves as the entry point of the Flutter mobile application, presenting a minimal interface with university email and password fields for returning users. New users are directed to a registration flow that collects email address, full name, roll number for campus students, and password, with the form adapting dynamically based on the selected tenant type to show or hide campus-specific fields. Following registration, an OTP verification screen is presented as an intermediate step, delivering a six-digit code via EmailJS for

identity confirmation. Subsequent logins are processed through the three-layer authentication system, evaluating identity, flag states, and role before routing to the appropriate application shell, with all error states surfaced inline without full-page navigation.

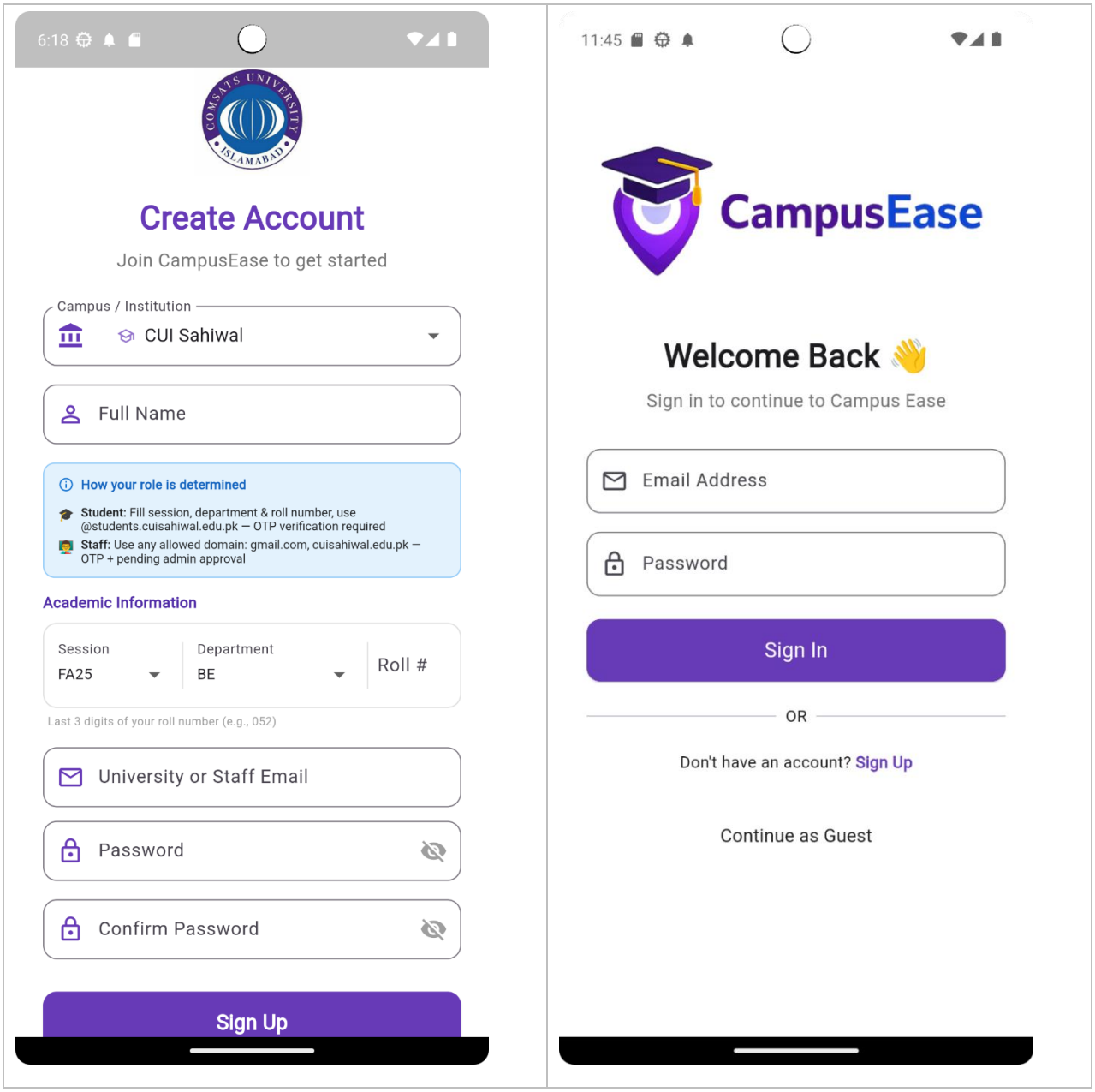

Figure 5.1:Registration and Login Screen

5.3.1.2 Home Dashboard

The Home Dashboard **Figure 5.2** presents a categorised overview of recent lost and found item postings, active announcements, and pending notifications. A bottom navigation bar provides access to the primary modules: Home, Report Item, Incidents, Announcements, and Profile. A floating

action button enables rapid item report submission. Push notification count badges on the notification icon provide at-a-glance awareness.

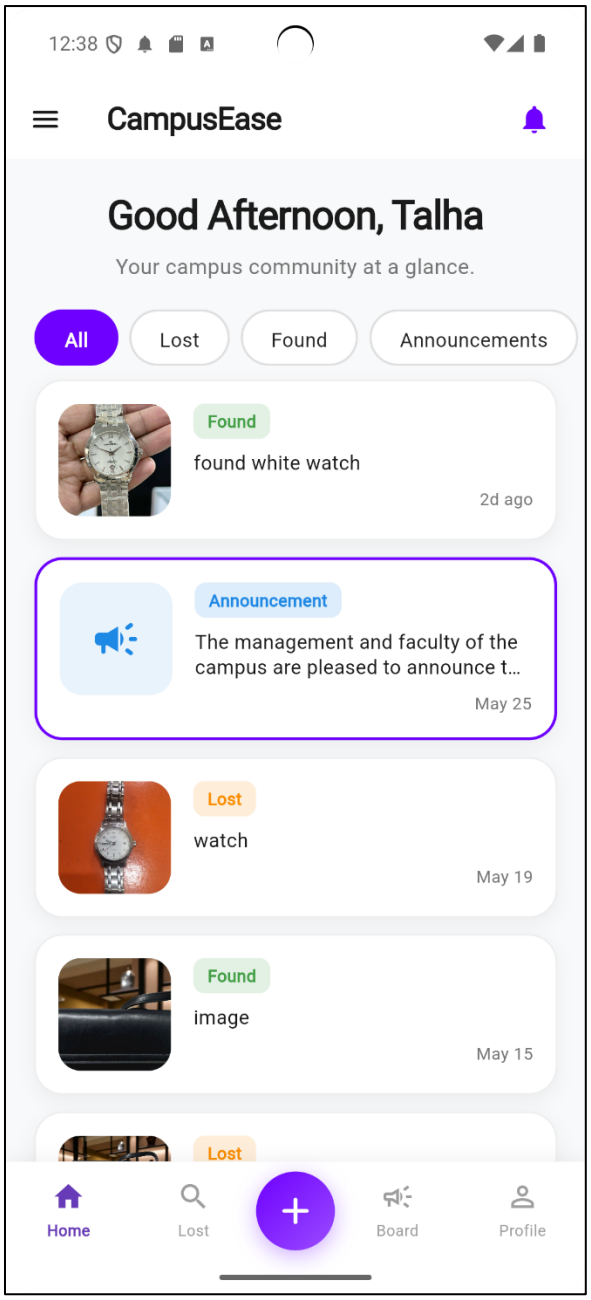

Figure 5.2:Home Screen

5.3.1.3 Reporting Screen

The reporting screen, shown in Figure 5.3, provides a unified form for Lost, Found, and Incident submissions accessed through a top-level type toggle. Item and incident categories are loaded dynamically from the active tenant's configuration in Firestore, ensuring that the category options

presented reflect the institution's specific operational taxonomy without requiring an application update when categories are added or modified by an administrator. The form organises inputs into sections for item details, dynamic category and subcategory selection, location, date, and contact preferences, with real-time field validation preventing submission with incomplete or invalid data.

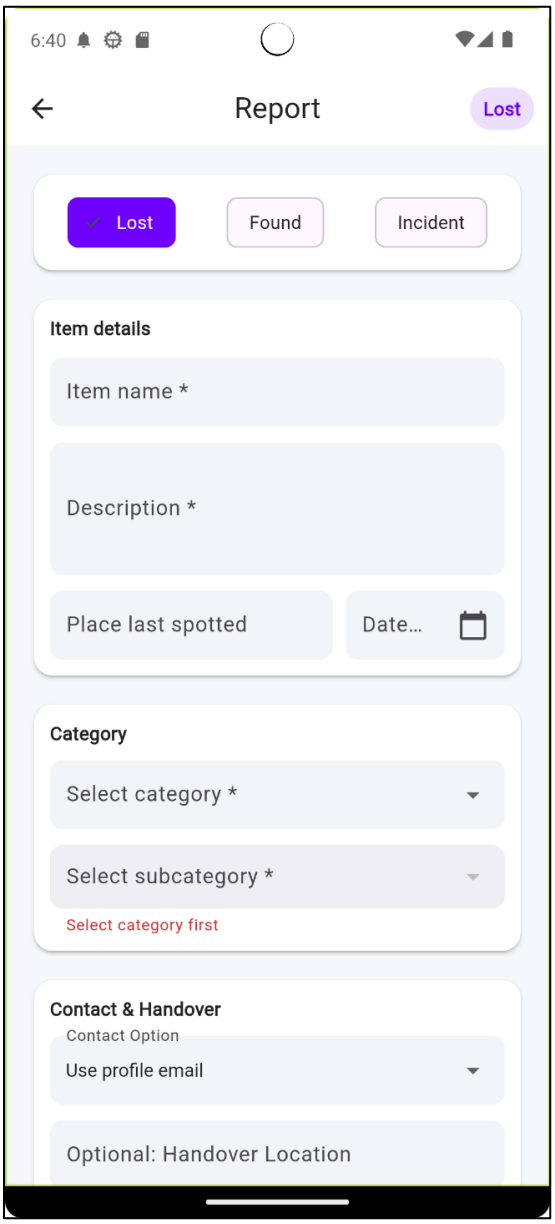

Figure 5.3:Reporting Screen

5.3.1.4 Item Detail Screen

The item detail screen, shown in Figure 5.4, displays the full item record including the uploaded image, title, description, category badge, reported date, and location. For relevant posts, the AI suggestions shown to users, redirected to this screen to choose an option, display each matched

counter-item's details and similarity score percentage in a horizontal card list. Action buttons are rendered conditionally based on item type and ownership: a Claim This Item button is shown to users viewing a found item they wish to claim, while an I Found This button is presented on lost item posts to users who have located the item. A comments section below the item details allows users to leave publicly visible notes on the post, supporting community-driven item recovery. Items posted by office staff are indicated with a distinct badge reflecting the postedByOffice flag stored in Firestore.

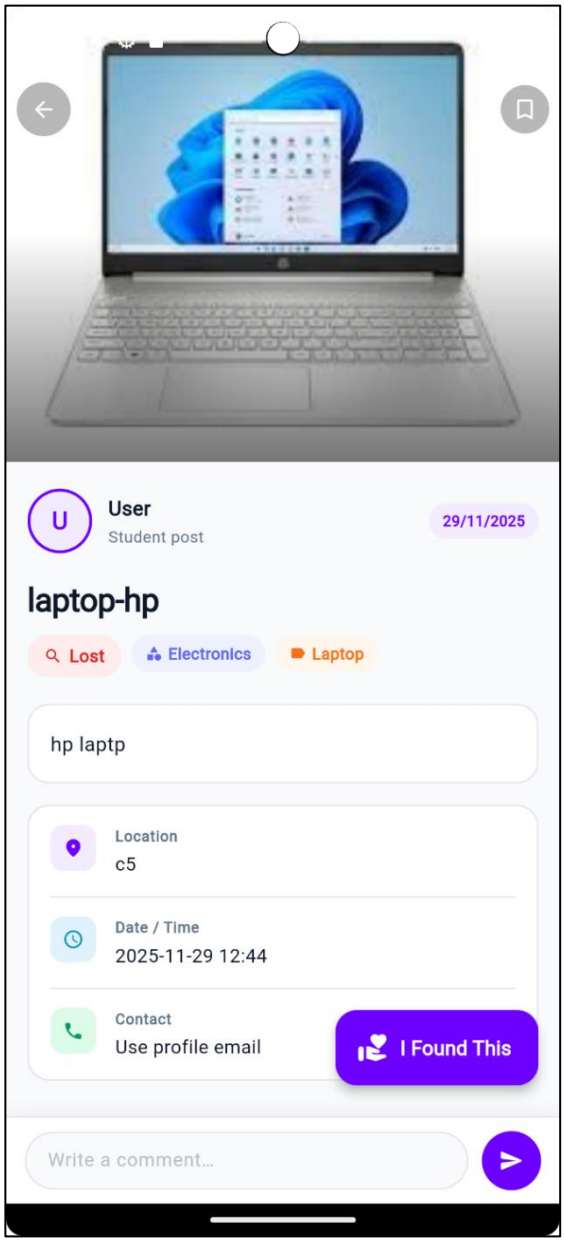

Figure 5.4:Item Detail Screen

5.3.1.5 Claim Submission Form

The claim form Figure 5.5 is presented when a user taps 'I Found This' on a Lost post or 'Claim Item' on a Found post. The form collects full name, WhatsApp contact number, date and time, location, an optional free-text message, an optional proof image upload via Cloudinary, and text answers to any custom ownership questions set by the post owner. Submission triggers ClaimProvider.createClaim() which writes the atomic batch to Firestore.

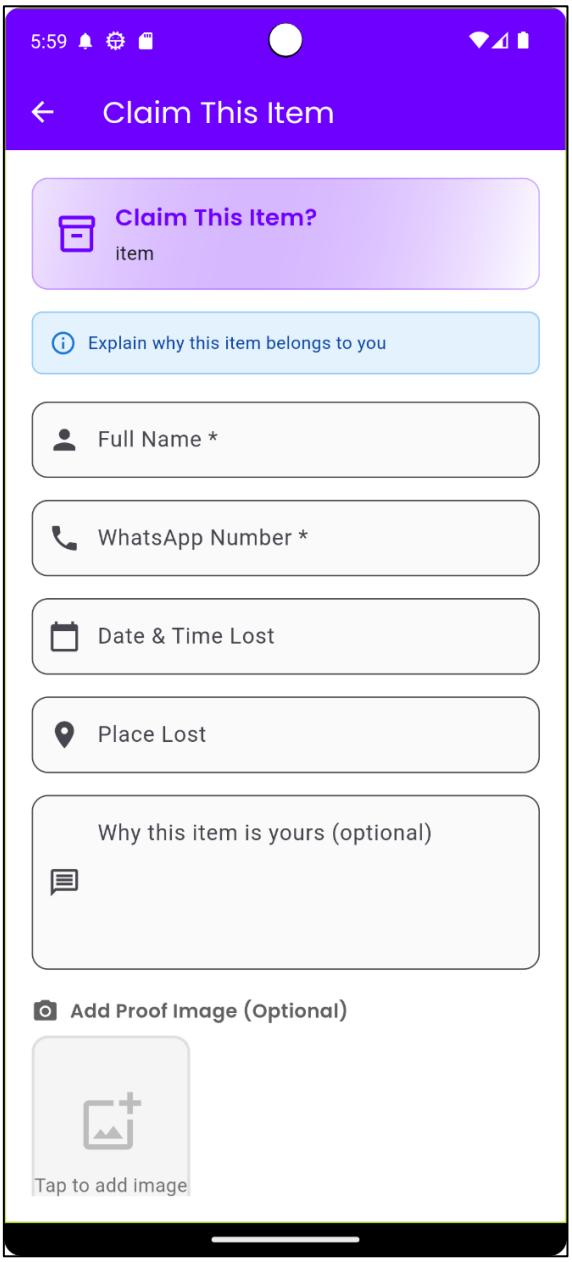

Figure 5.5:Claim Submission Form

5.3.1.6 Holder Claim Management Screen

The Holder Claim Management screen (Figure 5.6) is shared by student holders and office staff to manage the claim workflow from the post owner's perspective. It displays submitted claim details—including ownership answers, proof images, and claimant contact info—to facilitate informed acceptance or rejection decisions. Accepted claims progress seamlessly through meetup scheduling, OTP generation at physical handover, and final confirmation. If the standard flow encounters issues, users can utilize fallback mechanisms like rate-limited OTP regeneration, meetup rescheduling, or reporting a no-show/dispute to escalate the claim to an administrator.

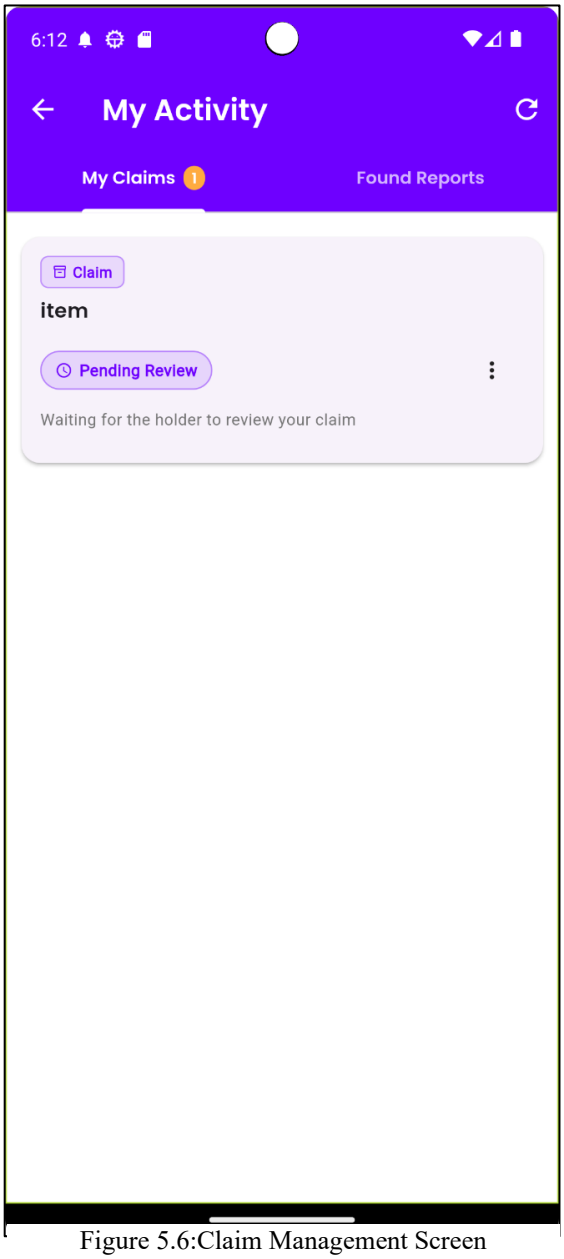

Figure 5.6:Claim Management Screen

5.3.1.7 Claimant Claim Detail Screen

The Claimant Claim Detail screen, shown in Figure 5.7, presents the claimant's view of an active claim, displaying the current state machine stage, proposed meetup details once scheduling has been initiated, and an OTP entry field that becomes interactive only at the otp_pending stage and remains blocked at all others. A cancel button is available from the pending stage through to meetup confirmation, after which cancellation is blocked to protect handover integrity. Where the meetup cannot be attended after otp_pending has been reached, a reschedule option is surfaced subject to the same attempt limit enforced on the holder side, and a report issue control is available to escalate the claim to an administrator if rescheduling does not resolve the situation.

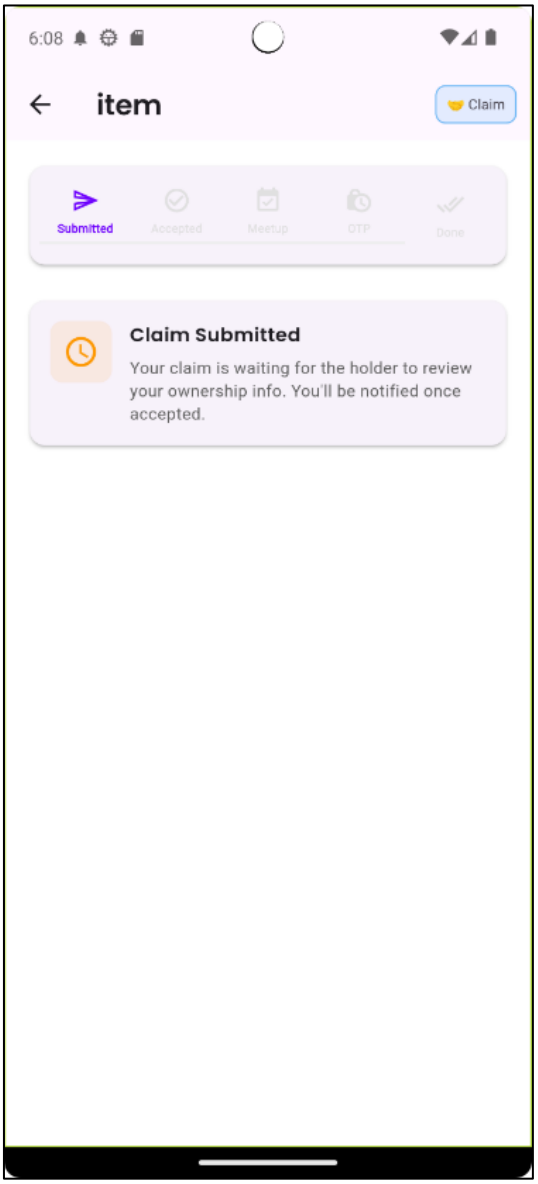

Figure 5.7:Claim Detail Screen

5.3.2 Admin Panel Interface

5.3.2.1 Admin Panel Dashboard

The React admin dashboard **Figure 5.8** presents a sidebar navigation with sections for Lost & Found Reports, Claims, Incidents, Users, Announcements, and Analytics. Key performance metrics (active items, pending claims, open incidents, user count) are presented as KPI widgets. The AI Matches section allows administrators to review suggested item pairs with similarity scores.

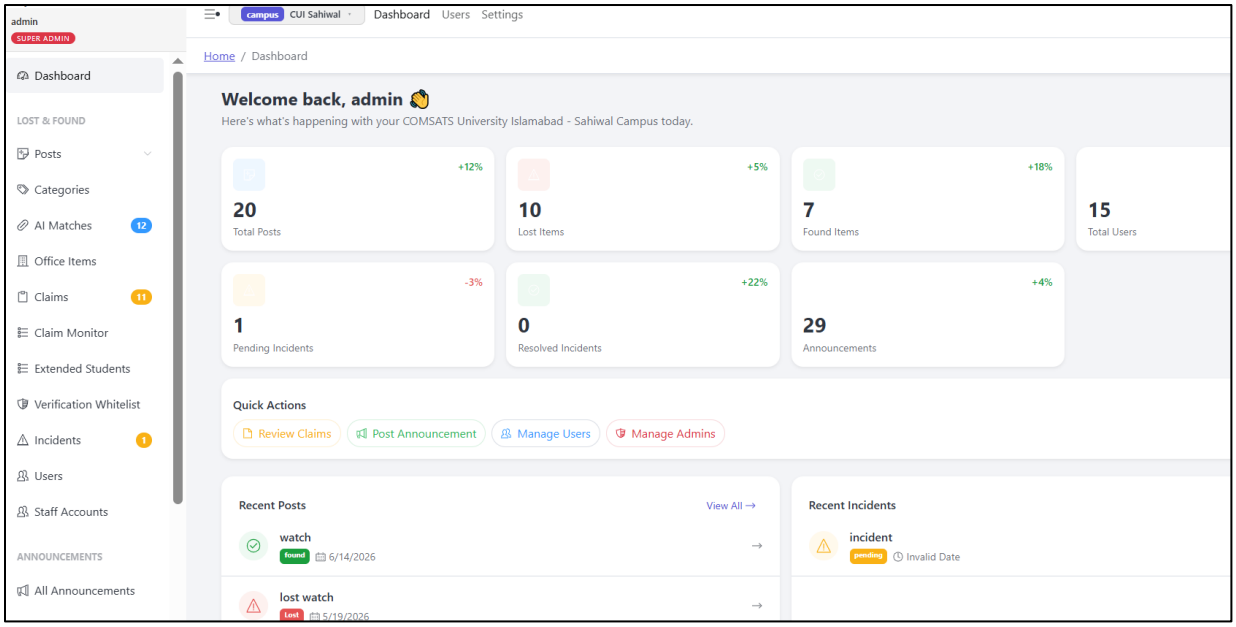

Figure 5.8:Admin Dashboard

5.3.2.2 User Management Screen

The user management screen provides administrators with a comprehensive interface for viewing, filtering, and managing all registered accounts within the active tenant. Users are presented in a tabular layout displaying name, email, role, registration date, and current flag states (active, verified, approved), enabling at-a-glance identification of accounts requiring action. Pending staff applicants and extended students in semesters 9 and 10 are surfaced with highlighted status indicators and inline approve and reject controls, which trigger automated EmailJS notifications to the affected user upon action. Each user entry is expandable into a full activity profile that aggregates the user's submitted posts, claims, incident reports, and AI match interactions into a tabbed interface, with each entry clickable and navigating directly to the corresponding detail view within the relevant module. This consolidated profile reduces administrative overhead when investigating user behaviour across

modules and provides a single point of reference for approval decisions, dispute resolution, and account moderation, operationalising the flag-based access control model described in Section 4.1.2 through directly actionable interface controls.

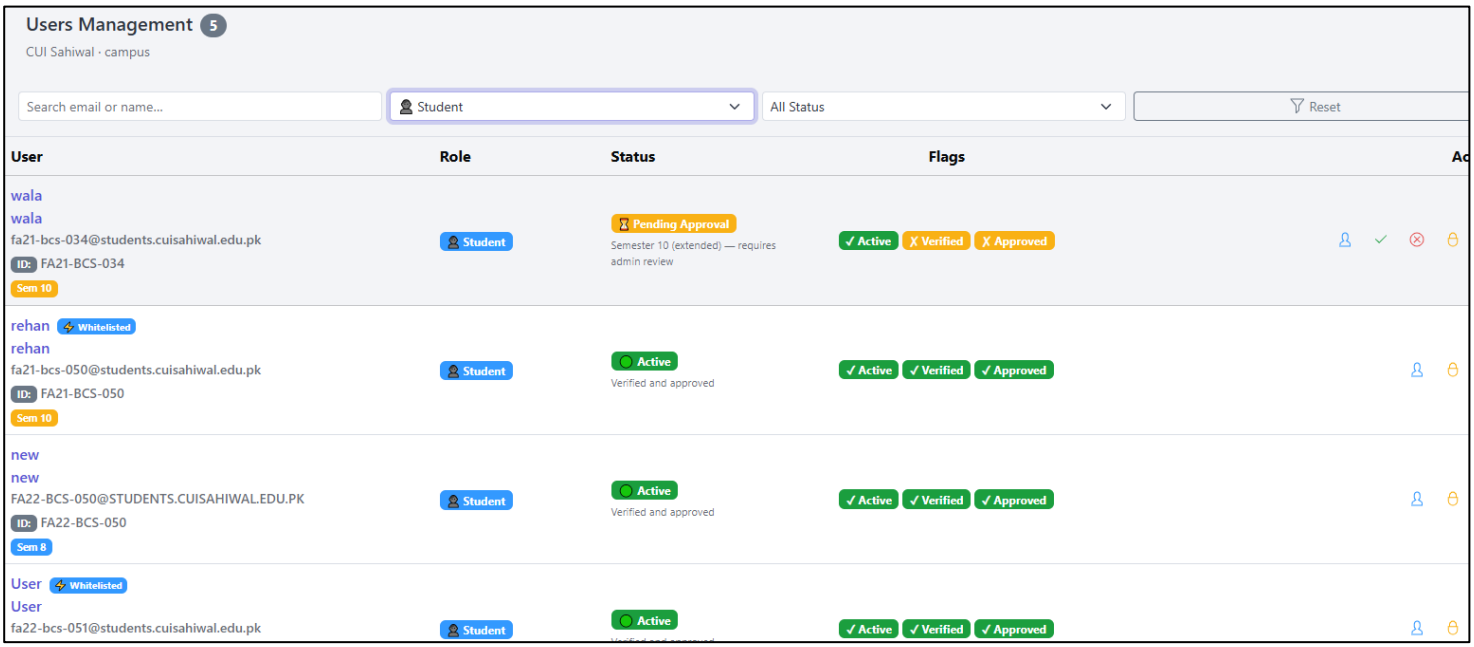

Figure 5.9: User Management Screen

5.3.2.3 Lost and Found Management Screen

The lost and found management screen provides administrators with a moderated view of all item reports submitted within the active tenant, categorised by type (lost or found), status, and submission date. Each item entry is expandable to reveal full details including the uploaded image, description, category, location, and submitting user profile. A dedicated AI Match Suggestions panel surfaces automatically generated match pairs produced by the on-device MobileNetV3 cosine similarity computation, displaying matched item images side by side with their combined weighted similarity score. Administrators can review each suggestion, mark it as confirmed or dismissed, and trigger a notification to the relevant users, adding a moderation layer that prevents low-confidence matches from reaching end users without review. Item categories displayed on this screen are dynamically configured per tenant: each campus, city, or district tenant may define a custom category set stored in the tenant document, which the Flutter application loads at runtime and reflects in both the submission form and the admin filter controls. The screen displays only items scoped to the currently active tenant, consistent with the platform-wide multi-tenant data isolation model.

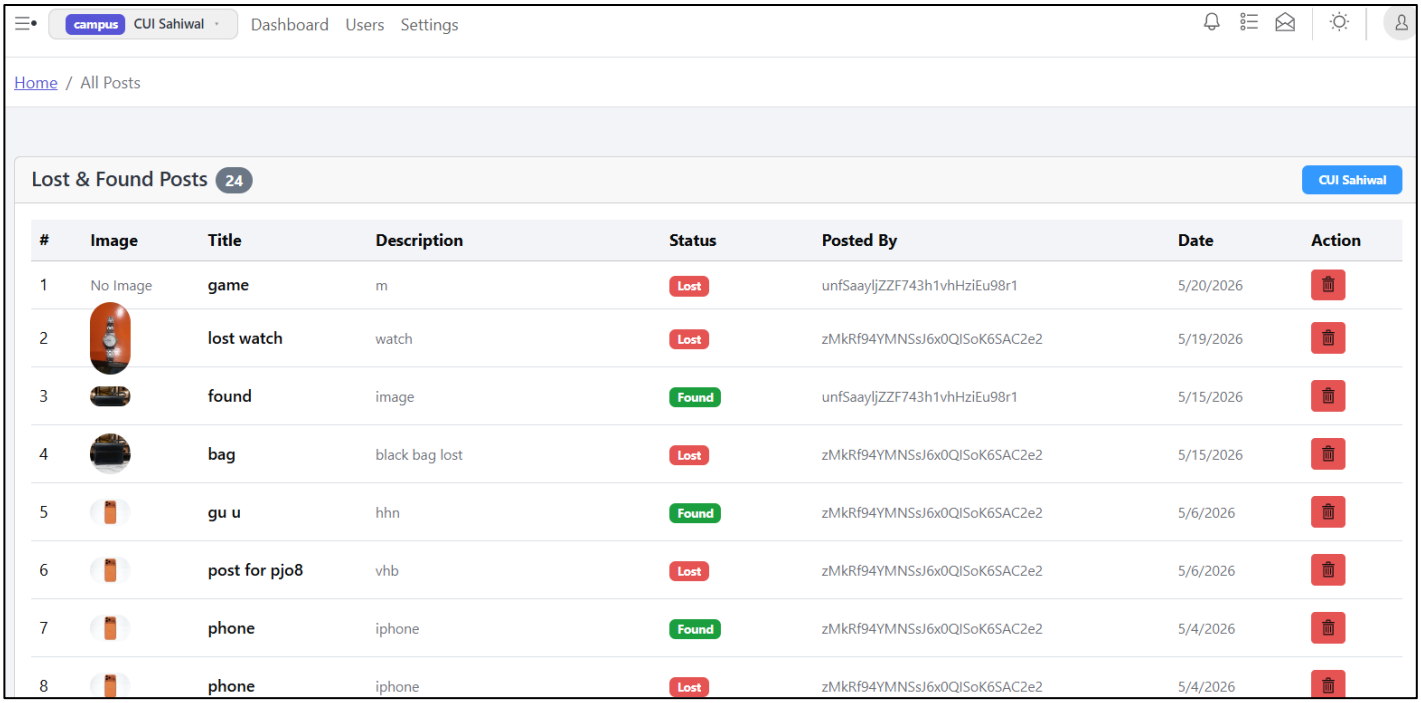

Figure 5.10: Lost and Found Management Screen

5.3.2.4 Claim Review and Verification Screen

The claim review and verification screen provides administrators with visibility into the peer-to-peer claim state machine described in Section 4.3, allowing oversight of all active and historical claim interactions within the tenant without direct involvement in the workflow. Each claim entry displays the current stage of the six-stage process (pending, accepted, meetup proposed, meetup confirmed, OTP pending, handover pending, or completed), the identities of the claimant and post holder, submitted ownership question answers, proof image, and contact details. Disputed claims are flagged distinctly and escalated to the administrator for resolution, at which point the administrator can review the full claim history, communicate a decision, and update the claim status directly from the interface. This screen demonstrates the system's audit capability, as every state transition recorded in the audit_log collection is accessible in chronological order, providing a transparent and accountable record of each item recovery interaction.

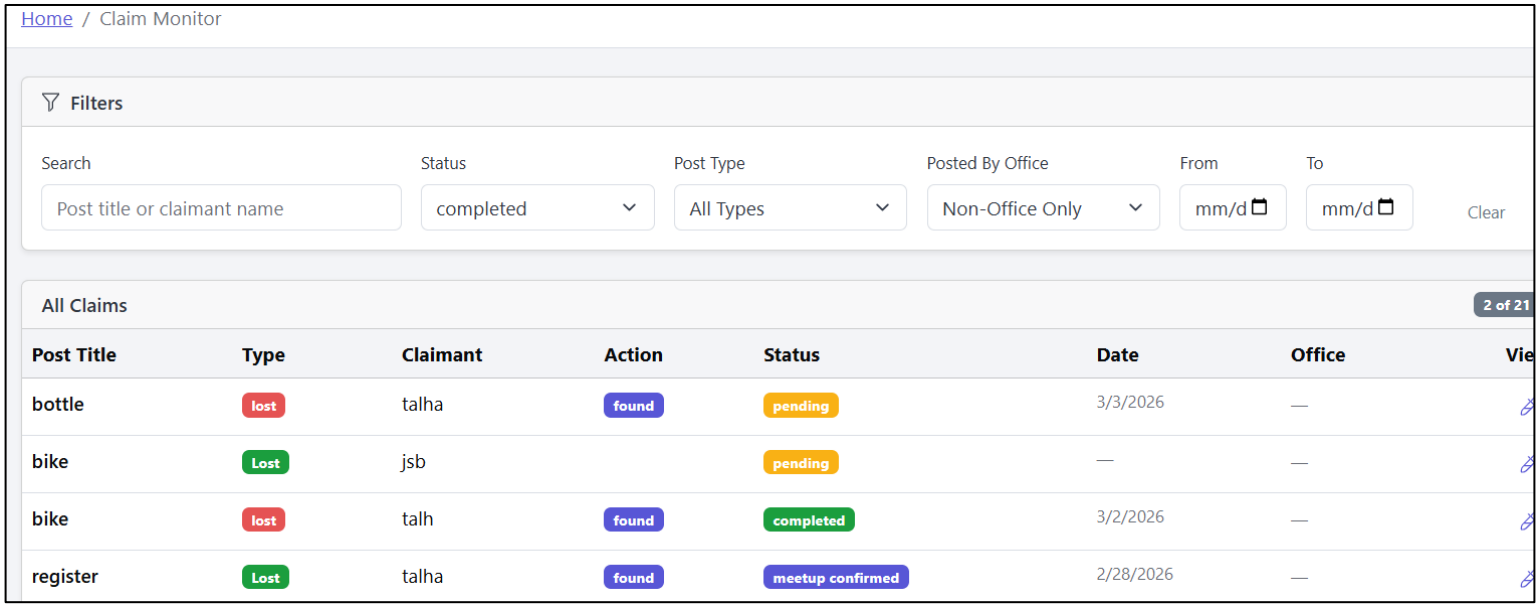

Figure 5.11:Claim Managment Screen

5.3.2.5 Incident Management Screen

The incident management screen consolidates all submitted incident reports within the active tenant into a structured, filterable interface that supports administrative triage, assignment, and resolution tracking. Each incident entry displays the user-provided title, category, description, location, and optional image alongside the AI-generated advisory fields produced by the Google Gemini 2.5 Flash Lite API, namely ai_category and ai_priority, which are rendered as distinct labelled badges to visually differentiate them from user-submitted values. Administrators can update the incident status through the workflow stages of pending, reviewed, and resolved, with each transition automatically queuing a push notification to the reporting user via the pendingPushNotifications Firestore collection. The screen supports filtering by status, category, and priority to enable efficient management of high-volume incident queues. The presentation of both user-provided and AI-suggested classifications within the same interface reflects the system's design philosophy of augmenting administrative decision-making rather than replacing it.

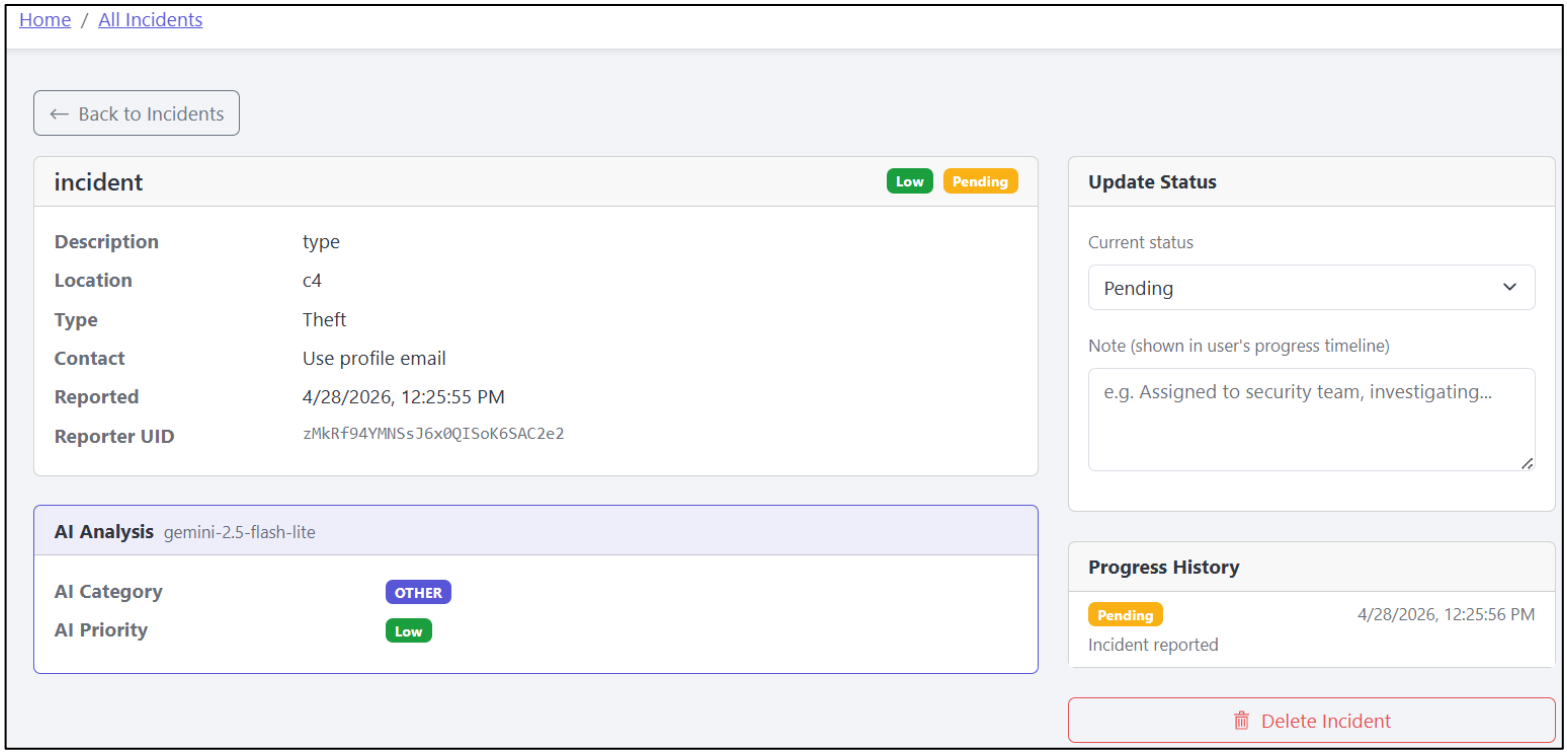

Figure 5.12: Incident Management Screen

5.3.2.6 Announcement Management Screen

The announcement management screen enables administrators to compose, schedule, and broadcast institutional announcements to targeted audiences within the active tenant. The creation interface provides fields for title, body text, optional image upload via Cloudinary, announcement type selection, and audience targeting, where campus tenants may target campus-wide, specific departments, academic batches, or events, and city or district tenants may target general public, emergency, event, or official notice audiences. Published announcements are listed in reverse chronological order with type badges, pinned status indicators, and delivery confirmation. Upon publication, the system writes a document to the pendingPushNotifications Firestore collection, which the Flutter NotificationService detects and relays to the OneSignal REST API for push delivery to all targeted devices. The listing view also allows administrators to edit, pin, or delete existing announcements, providing full content lifecycle management from a single interface.

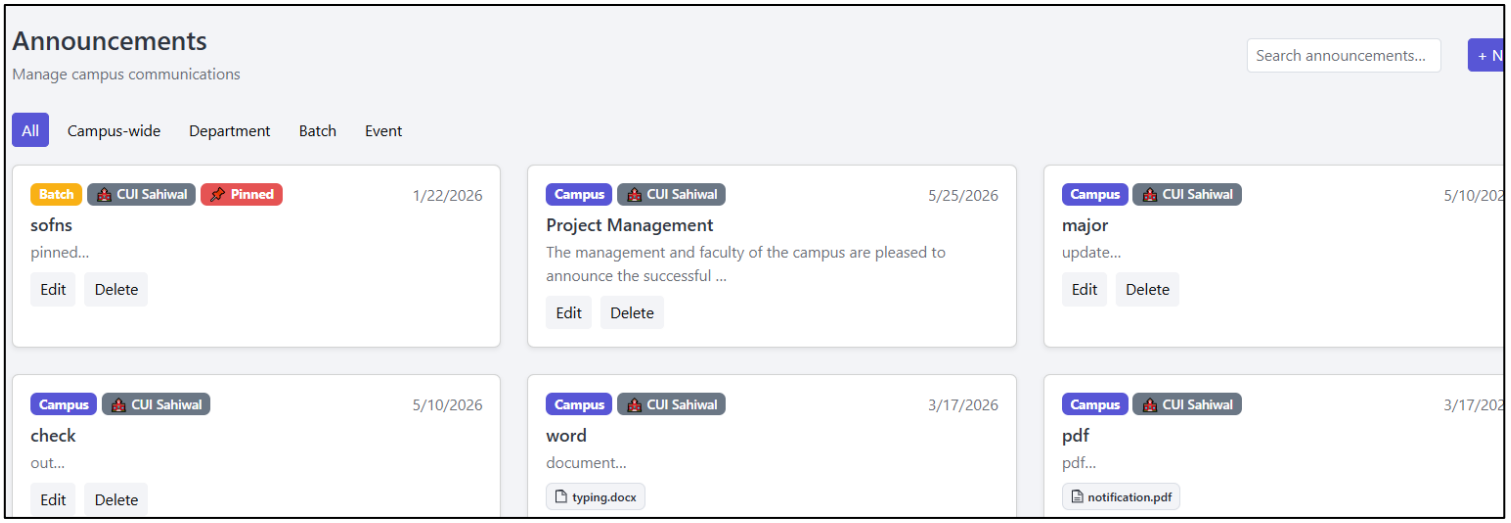

Figure 5.13: Announcement Management Screen

5.3.2.7 Tenant Management Screen

The tenant management screen is accessible exclusively to super administrator accounts and provides the foundational governance interface for the CampusEase multi-tenant SaaS architecture. The screen presents all registered tenant institutions in a tabular listing with their name, short name, institution type (campus, city, or district), allowed email domain, active status, and creation metadata. The creation and editing form allows super administrators to define the institution type, configure the allowedDomain and studentDomain fields for email validation during student registration, and activate or deactivate the tenant. The TenantSwitcher component embedded in the navigation header allows the super administrator to change the active tenant context at any point, with the selected tenant persisted in localStorage to maintain session continuity across page refreshes. All subsequent administrative operations performed after a tenant switch are automatically scoped to the newly selected institution, demonstrating the practical implementation of the multi-tenant isolation model described in Section 5.4.

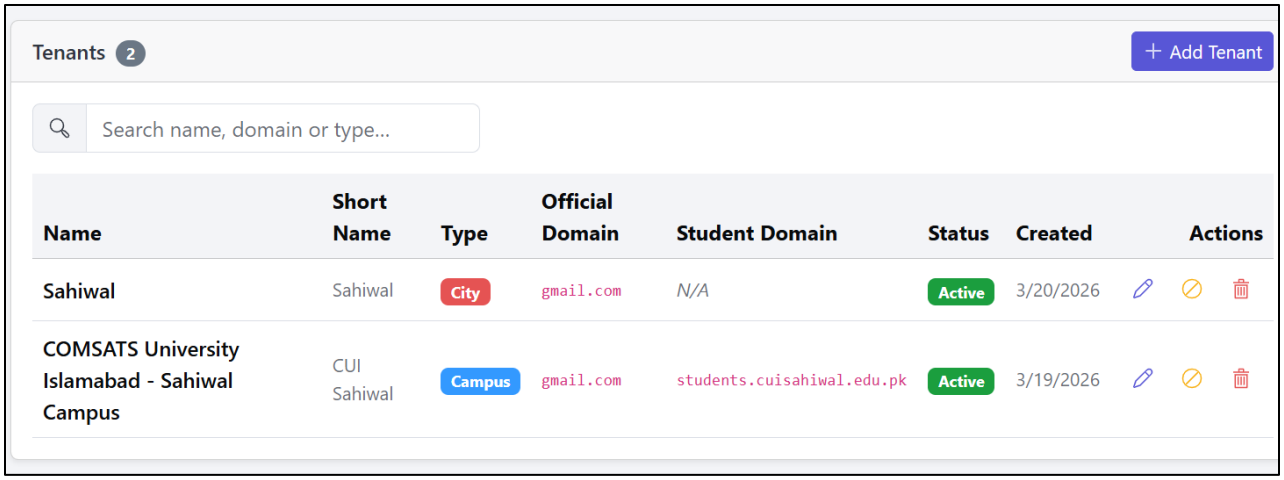

Figure 5.14: Tenant Management Screen

5.3.3 Deep Linking and Shareable Content

The CampusEase mobile application supports deep linking to allow users to share individual lost and found posts, announcements, and incident reports via external channels such as WhatsApp and SMS. Each shareable item generates a URL using a custom URI scheme for direct in-app routing and an HTTPS fallback URL for recipients who do not have the application installed. The HTTPS fallback renders a guest-accessible preview of the item within the application, displaying key details without requiring authentication, and presents a prompt to register or log in for full interaction. All deep link routes are tenant-safe, meaning the link encodes the tenant identifier alongside the document identifier, ensuring that recipients are routed to the correct institutional context upon opening the application. Tenant-scoped Firestore security rules prevent cross-tenant document access even if a link is shared across institutional boundaries.

5.3.4 CampusEase Web Platform Interface

The CampusEase web platform provides a publicly accessible interface that complements the Flutter mobile application and React admin dashboard as the third client layer of the system. The landing page presents a structured overview of the platform including its core modules, supported institution types, and technology stack, serving as the primary point of discovery for prospective institutional adopters and end users. A prominently displayed download section presents the APK installation option when the administrator toggle is enabled in Firestore, allowing users to install the application directly without requiring app store availability. The Privacy Policy and Terms and Conditions pages

are hosted as static routes on the same domain, satisfying the legal disclosure requirements associated with user data collection and authentication. For users arriving via a shared deep link without the application installed, the web platform renders a lightweight guest preview page displaying the referenced item or announcement, maintaining the utility of shared content across both installed and non-installed user contexts.

5.4 Mobile Application Functional Logic

The Flutter mobile application was updated to reflect the flag-based authentication architecture. The signup screen for campus tenants now loads session options dynamically from the Firestore sessions collection, replacing the previous hardcoded list. Sessions are displayed in seq-order and filtered to isActive=true, ensuring that closed sessions are not presented as options to new registrants. The selected session's semester value determines the initial approved flag as described in Section 5.1.2.

A Pending Approval screen was added to the authentication flow, presented to users whose approved flag is false after OTP verification. The screen displays a confirmation message that the account is under administrative review, a contact reference for the campus administration office, and a logout option. The screen uses a Firestore real-time listener on the user document to automatically advance the user to the main application shell when approval is granted, without requiring manual app restart.

5.4.1 Role-Based UI Navigation

Each of the four functional roles is mapped to a distinct navigation shell. Students and staff access the campus shell with the full bottom navigation bar: Home, Lost and Found, Incidents, Announcements, and Profile. Office users access the office shell with Post Item, My Posts, Claims Management, and Profile tabs; the Incidents and Announcements tabs are absent as office staff are not the target audience for these modules. Public users access the city or district shell with city-specific announcement types and without campus-specific features such as roll number display and session-based filtering.

Flag-based conditional UI rendering is applied throughout all shells. If a user's active flag is set to false during an active session as a result of an administrative action, the Firestore real-time listener on the user document detects the change and redirects the user to a blocked account screen without requiring a logout-login cycle. This ensures that administrative actions take effect in near-real-time on active devices.

5.4.2 Deep Link Routing and Guest Preview

The deep linking system handles two routing scenarios within the Flutter application. When an authenticated user opens a deep link, the application resolves the encoded tenant and document identifiers, verifies tenant membership, and navigates directly to the relevant detail screen. When an unauthenticated user opens a link via the HTTPS fallback, the application renders a read-only guest preview of the referenced item without initiating an authentication session or performing any Firestore write operations, and presents a prompt to register or log in. The originally requested destination is stored and resumed upon successful authentication via the three-layer authentication flow.

The Vercel-hosted web platform acts as the HTTPS fallback layer for all shared links. When a link is opened on a device without the application installed, the Vercel route extracts the tenant and document identifiers from the URL and renders the guest preview interface. Where the application is installed, the custom URI scheme takes precedence and routes the user directly into the Flutter app without passing through the web layer. The web platform and Flutter application share the same URL structure, ensuring seamless transition between fallback and in-app routing without duplication of routing logic.

5.4.3 OTP Recovery and Claim Deadlock Prevention

The OTP recovery mechanism addresses edge cases in the claim handover workflow where the scheduled meetup cannot be completed as planned. OTP regeneration is restricted to the holder and is rate-limited to prevent repeated generation that could undermine the security of the handover process. Meetup rescheduling is available to both parties and is limited to a configurable number of attempts per claim to prevent indefinite postponement. All recovery actions are written to the claim document with a timestamp and the acting user's identifier, maintaining the integrity of the audit trail. If neither recovery option resolves the situation within the permitted attempts, the claim is automatically flagged for administrator review, and a document is written to the disputes collection with a system-generated reason indicating recovery exhaustion. This mechanism prevents genuine claims from entering a permanent deadlock state while preserving the accountability guarantees of the six-stage workflow.

5.5 Multi-Tenant Architecture Implementation

A distinguishing architectural achievement of CampusEase is its production-grade multi-tenant SaaS implementation on a single shared Firebase Firestore database. Rather than provisioning a separate database instance per institution, data isolation is enforced at the query layer: every document in the posts, announcements, incidents, users, and sessions collections carries a tenant_id field, and every Firestore read or write operation is scoped exclusively to the authenticated user's tenant identifier. Three institution types are supported: campus (with student email domain validation and roll number verification), city, and district. The signup flow dynamically adapts its interface based on tenant type, with the studentDomain field physically absent from city and district tenant documents rather than stored as an empty string, allowing the form to detect the institution type without additional conditional logic.

The React admin panel provides a TenantSwitcher component enabling super administrators to switch between tenants without logging out, with the selected tenant persisted in localStorage to maintain correct scoping across page refreshes. Non-super-admin roles receive a read-only tenant badge confirming their assigned institution. Composite Firestore indexes on (tenant_id ASC, createdAt DESC) maintain query performance as document volumes scale across all tenants. Staff accounts are provisioned using a secondary Firebase app instance to prevent the admin session from being replaced during account creation, and a paginated backfill toolset handles legacy field migration using batch writes of 400 documents per operation. The dynamic category system further extends the multi-tenant model to operational content: each tenant document may define a custom incidentCategories array that the Flutter application reads at runtime, with category changes taking effect immediately on the next form load without requiring an application update.

CHAPTER 6
TESTING AND EVALUATION

6 Testing and Evaluation

Comprehensive testing of CampusEase was conducted across multiple testing strategies to validate functional correctness, performance, security, and user experience. The testing strategy encompassed manual testing (system, unit, functional, and integration testing) and automated testing using Flutter testing tools and Firebase console monitoring.

6.1 Manual Testing

Manual testing was conducted systematically by the development team using physical Android devices (Samsung Galaxy A52, Xiaomi Redmi Note 11) and Android emulators via Android Studio. Test execution followed structured test case specifications with clearly defined inputs, expected outputs, and pass/fail criteria.

6.1.1 System Testing

System testing evaluated the CampusEase platform as a complete, integrated system against the specified functional and non-functional requirements. Test scenarios encompassed full end-to-end user journeys from registration through item reporting, AI matching, claim verification, and notification receipt. The admin panel was tested in parallel to verify that user-initiated actions produced the correct administrative responses and that administrative actions (claim approval, incident status updates, announcements) correctly propagated to the mobile client.

System testing confirmed that the three principal modules Announcements, Lost and Found Management, and Incident Reporting operate coherently as a unified system without inter-module data conflicts or navigation breakdowns. Role-based access control was verified across all three user roles, confirming that students cannot access administrative functions and that office staff cannot perform user management actions reserved for administrators.

6.1.2 Unit Testing

Unit testing was conducted on individual Flutter widgets, service classes, and utility functions. Key units tested included: the MobileNetV3 embedding generation function (verifying output dimensionality of 1280 and value range); the cosine similarity computation function; ClaimProvider state machine guards (_assertOwner, _assertStatus, ClaimStatus.cancellable list) confirming correct

exceptions are thrown for invalid transitions; OtpProvider.generateOtp() producing a 6-digit string written to Firestore and OtpProvider.verifyOtp() correctly validating and rejecting inputs; MeetupModel.isWithinWindow() returning the correct boolean for times within and outside the allowed window; AuditService entry structure and field completeness; and form validation logic in ClaimOrFoundFormPage. Flutter's built-in test package was used to write widget tests for HolderClaimManageScreen and ClaimDetailScreen confirming correct action button visibility per claim status.

6.1.3 Functional Testing

Functional testing validated each functional requirement against its specified behaviour. The following test case table presents a representative sample of functional tests executed:

Table 6.1:Functional Test Cases & Results

TC ID	Test Description	Input	Expected Output	Result
TC-01	User Registration with Valid University Email	Email: student@comsats.edu.pk, Password: ValidPass123!	Account created; OTP sent to email; user redirected to dashboard	PASS
TC-02	User Registration with Non-University Email	Email: user@gmail.com, Password: ValidPass123!	Error: 'Please use your university email address'	PASS
TC-03	Lost Item Report Submission (All Fields)	Title: Black Wallet, Category: Other, Image: wallet.jpg	Report stored in Firestore; embedding generated; user notified	PASS
TC-04	Lost Item Report Missing Image	All fields complete except image upload	Form validation error: 'Please upload an image of the item'	PASS
TC-05	AI Match Detection Visually Similar Items	Lost item: blue pen image; Found item: same pen photographed differently	Match suggestion created with similarity score >= 0.70	PASS

TC ID	Test Description	Input	Expected Output	Result
TC-06	AI Match Detection Dissimilar Items	Lost item: laptop; Found item: water bottle	No match suggestion created (similarity < 0.70)	PASS
TC-07	Claim Submission action=claim on Found Post	User taps Claim Item; fills ClaimOrFoundFormPage with valid details and ownership question answers	Claim document created at posts/{postId}/claims/{claimId} with status=pending; user_claims_index entry written; audit event claimSubmitted logged; holder push notification received	PASS
TC-08	Claim Submission action=found on Lost Post	User taps I Found This on a Lost post; submits form	Claim created with action=found and status=pending; same batch write and notifications as TC-07	PASS
TC-09	Holder Accepts Claim Atomic Batch Write	Holder opens HolderClaimManageScreen and taps Accept	claim.status=accepted; post.status=under_claim; post.primaryClaimId set all in one atomic Firestore batch. Claimant notified.	PASS
TC-10	Holder Rejects Claim with Reason	Holder taps Reject with reason 'Wrong description'	claim.status=rejected; post.status reverts to active; primaryClaimId deleted; claimant notified with reason text	PASS
TC-10b	Meetup Proposal and Counter-Proposal	Holder proposes meetup; claimant counter-proposes with new time	Status cycles meetup_proposed → meetup_proposed (counter); holder notified; loop continues	PASS

TC ID	Test Description	Input	Expected Output	Result
			until claimant accepts → meetup_confirmed	
TC-10c	OTP Generation Displayed Holder-Side Only	Holder generates OTP at meetup (status=meetup_confirmed)	Status advances to otp_pending; 6-digit OTP displayed on holder's screen; claimant OTP entry field activates; OTP not visible on claimant screen	PASS
TC-10d	OTP Verification Correct OTP	Claimant enters correct OTP on ClaimDetailScreen	OtpProvider.verifyOtp() succeeds; status advances to handover_pending; holder's confirm handover button becomes active	PASS
TC-10e	OTP Verification Wrong OTP	Claimant enters incorrect OTP	Error message shown; status remains otp_pending; no state change in Firestore	PASS
TC-10f	Handover Confirmation Completes Claim	Holder taps Confirm Handover	claim.status=completed; post.status=resolved; audit event handoverComplete logged; both parties receive push notification	PASS
TC-10g	Claimant Cancels Before OTP Stage	Claimant cancels from accepted status	claim.status=cancelled; post.status reverts to active; cancellation blocked if status is otp_pending	PASS
TC-10h	Dispute Raised by Holder	Holder taps Raise Dispute from HolderClaimManageScreen	claim.status=disputed; disputes/{disputeId} document created; appears in React admin panel for review	PASS

TC ID	Test Description	Input	Expected Output	Result
TC-11	Incident Report Submission	Category: Maintenance, Description: 'Broken fire exit door, Block B'	Incident stored; routed to facilities admin; reporter notified	PASS
TC-12	Announcement Broadcast by Admin	Title: 'Campus Closed Friday', Body: announcement text	Announcement in Firestore; push notification received on all devices within 30 seconds	PASS
TC-13	Search Items by Category Filter	Filter: Electronics; user has items in multiple categories	Only Electronics items returned in results	PASS
TC-14	Admin User Role Update	Admin changes user role from Student to Staff	User's Firestore role field updated; user gains staff permissions on next login	PASS
TC-15	Unauthorised Admin Panel Access (Student)	Authenticated student attempts direct URL access to admin dashboard	Access denied; redirected to login with insufficient permissions error	PASS

6.1.4 Integration Testing

Integration testing validated the end-to-end data flows between system components. Critical integration test scenarios included:

- Flutter App Firebase Firestore Integration: Verified that item reports and claim submissions via Flutter forms correctly appear as Firestore documents at the expected paths with all fields and correct data types.

- Claim State Machine Batch Write Integrity: Confirmed that all status-transition batch writes atomically update the claim document, the parent post document, and the user_claims_index entry in a single commit, with no partial states observable under simulated network interruption.

- OtpProvider State Gate Enforcement: Verified that OtpProvider.verifyOtp() throws the expected exception when called with a claim whose status is not otp_pending, confirming state machine guard integrity.

- NotificationService isOwnerView Routing: Confirmed that push notifications dispatched on claim status changes carry the correct isOwnerView flag, routing holders to HolderClaimManageScreen and claimants to ClaimDetailScreen on notification tap.

- Flutter NotificationService OneSignal Integration: Verified that announcement creation by the React admin panel writes a pending document to pendingPushNotifications, which the Flutter NotificationService detects and relays to the OneSignal REST API, resulting in push notification delivery to all target devices within 30 seconds.

- Flutter App TFLite Model Firestore Integration: Confirmed that the on-device MobileNetV3 model produces embeddings that are correctly serialised and stored in Firestore, and that the Flutter PostsProvider retrieves these embeddings and produces mathematically valid cosine similarity scores entirely on-device, with no server-side computation involved.

- React Admin Dashboard Firestore Integration: Verified that admin actions (claim approval, incident status update, announcement creation) performed in the React dashboard correctly update Firestore documents, which trigger mobile client real-time listeners and, where applicable, write to the pendingPushNotifications queue for the Flutter NotificationService to relay to OneSignal.

- Deep Link Routing: Verified that shared post and announcement links resolve correctly to the intended detail screen for authenticated users, and that the HTTPS fallback renders a guest preview for unauthenticated recipients without initiating a Firestore write operation.

- Deep Link Tenant Isolation: Confirmed that a deep link generated for a document in Tenant A does not grant access to that document when opened by a user authenticated under Tenant B, with the Firestore security rule returning a permission denied error and the application displaying an appropriate message.

- Password Reset across Institutional Email: Verified that the Vercel serverless function generates a valid Firebase Admin SDK reset link and that the EmailJS delivery successfully reaches institutional email addresses that reject Firebase Authentication's default reset email sender domain.

OTP Regeneration Rate Limiting: Confirmed that the OTP regeneration control is disabled after the maximum permitted attempts and that a new OTP generated after expiry correctly invalidates the previous value in Firestore before the claimant can attempt entry.

- Meetup Reschedule Attempt Limit: Verified that the reschedule option is blocked after the configured maximum attempts and that the report issue fallback is surfaced automatically to both parties, with a dispute document correctly written to the disputes collection.
- Dynamic Category Loading: Confirmed that a custom incidentCategories array defined in the tenant document is correctly loaded by the Flutter application at form render time and that deactivation of a category in the admin panel removes it from the submission form on the next load without requiring an app restart.

6.2 Automated Testing

Flutter Testing Tools

Flutter's built-in test framework was used to implement widget tests for key UI components. Widget tests were written for the Registration Form (validating form field rendering and validation logic), the Item Report Form (verifying stepper navigation and field state management), and the AI Match Results Card component (confirming correct display of similarity score and item details). The flutter_test package's WidgetTester API was used to simulate user interactions and assert expected widget states. Integration tests were written using the flutter_driver package to simulate end-to-end user flows on a connected device.

Firebase Console Monitoring

The Firebase Console was used as a monitoring and verification tool throughout development. Firestore document monitoring was used to verify that pendingPushNotifications documents were correctly written by the React panel and subsequently marked as sent by the Flutter NotificationService, confirming end-to-end notification queue operation. The Firestore Rules Simulator was used to validate security rule configurations against representative read/write scenarios for each user role. Firebase Authentication monitoring confirmed successful user creation and authentication events. Performance Monitoring was enabled to capture app startup time, Firestore query latency, and network request durations under realistic conditions.

CHAPTER 7

CONCLUSION

7 Conclusion and Future Work

7.1 Conclusion

CampusEase is a comprehensive AI-enhanced, multi-tenant campus operations platform developed to address inefficiencies in manual service management at COMSATS University Islamabad, Sahiwal Campus. It integrates Announcements, Lost and Found Management, and Incident Reporting into a unified authenticated ecosystem, improving communication, accountability, and operational efficiency.

A key contribution of the system is its production-grade multi-tenant SaaS architecture built on a single Firestore database using tenant-based isolation, incorporating a structured authentication and access control model that combines identity verification, flag-based user state management, and role-based routing alongside approval workflows and session management suitable for real-world campus governance. The system was further extended with a supporting web platform deployed on Vercel serving as the public-facing entry point, APK distribution endpoint, deep link fallback layer, and password reset handler, with tenant-configurable dynamic categories and guest preview support extending the platform's reach and adaptability beyond its core registered user base.

Technically, the system integrates on-device AI-based image similarity using TensorFlow Lite for privacy-preserving matching, and implements a secure multi-stage claim workflow with OTP-based physical verification and audit tracking to prevent fraud and ensure accountability. The design also emphasizes maintainability through simplified role management and layered authentication separation.

From a software engineering perspective[18], [19] , the project demonstrates effective application of Agile development practices and integration of multiple technologies, including Flutter, React, Firebase, and TensorFlow Lite, into a unified system architecture. All functional and non-functional requirements were successfully implemented and validated through comprehensive testing.

The system is fully deployed and operational at COMSATS University Islamabad, Sahiwal Campus, providing practical value to the campus community.

7.2 Future Work

7.2.1 Real-Time Camera-Based Item Detection

Integration of real-time object detection using YOLO (You Only Look Once) or a comparable detection model would enable users to point their camera at a found item and immediately query the lost items database for visual matches, without requiring the explicit image upload and report submission workflow. This would significantly reduce the friction of found item reporting and potentially increase the volume and timeliness of found item submissions.

7.2.2 Larger and More Accurate AI Models

The current MobileNetV3 implementation was selected for on-device deployment efficiency. Future work could explore fine-tuning the model on a domain-specific dataset of university campus item images to improve matching accuracy for the specific categories most commonly encountered in campus lost-and-found contexts. Alternatively, larger models (EfficientNet-B4 or ResNet-50) deployed via a cloud inference API with appropriate privacy controls could offer improved accuracy at the cost of network dependency.

7.2.3 Campus Security System Integration

Future integration with campus CCTV infrastructure via a video analysis pipeline could enable automated detection and reporting of found items and incidents from camera feeds, with AI-identified items automatically pre-populating the found item report form. This would position CampusEase as a component of a broader campus intelligence platform rather than a standalone reporting tool.

7.2.4 Improved Fraud Prevention

While OTP verification provides a baseline claim fraud prevention mechanism, future work could explore the incorporation of multi-factor ownership verification, including natural language processing analysis of claimant-provided ownership descriptions compared against item report details, and optional blockchain-based ownership attestation for high-value items.

7.2.5 Multi-Tenant Expansion and Analytics

The multi-tenant architecture is implemented and operational. Immediate future work includes: connecting the Flutter signup session selector to the live Firestore sessions collection (currently using a hardcoded list); completing the password reset flow; implementing a full student profile edit screen; and delivering the remaining office shell features. Longer-term, aggregated cross-tenant analytics dashboards and tenant-scoped admin role accounts are priority enhancements. The EmailJS

free tier (200 emails/month) should be replaced with a scalable email delivery service such as SendGrid or AWS SES for production deployment.

7.2.6 In-App Messaging and Community Features

The optional in-app chat feature identified in the functional requirements but deferred from the current implementation represents a high-priority enhancement. Secure, moderated direct messaging between item owners and finders, facilitated after a match suggestion is generated, would reduce reliance on admin-mediated claim processes for straightforward cases and improve overall item recovery velocity. Enhanced Flag-Based Permission Granularity: The current flag system uses three boolean flags to model user lifecycle states. Future work could extend this to additional verification dimensions, including phone_verified for two-factor authentication and department_verified for confirming departmental enrollment with registrar records. Each additional flag would be evaluated as an additional gate in the Layer 2 authentication check, providing progressively finer-grained access control without altering the role hierarchy. Audit Logging for Administrative Flag Changes: All flag modifications performed by administrators are currently reflected in push notifications and email records but are not captured in a structured audit log collection. A future enhancement would write an immutable audit entry to an admin_audit_log collection on every flag change, recording the administrator UID, affected user UID, field changed, old value, new value, and timestamp. This would provide a complete administrative action trail for institutional compliance and dispute resolution purposes. Bulk Administrative Operations: The current admin panel processes approval and rejection actions one user at a time. Future work could implement bulk selection and batch approval or rejection using Firestore batch writes applied to multiple user documents simultaneously. This would significantly reduce administrative overhead during peak enrollment periods when large numbers of extended students or staff applicants require simultaneous processing.

References

[1] W. M. Al-Rahmi *et al.*, "Integrating Technology Acceptance Model With Innovation Diffusion Theory: An Empirical Investigation on Students' Intention to Use E-Learning Systems," *IEEE Access*, vol. 7, pp. 26797–26809, 2019, doi: 10.1109/ACCESS.2019.2899368.

[2] B. A. Kumar and B. Sharma, "Context aware mobile learning application development: A systematic literature review," *Educ. Inf. Technol.*, vol. 25, no. 3, pp. 2221–2239, May 2020, doi: 10.1007/s10639-019-10045-x.

[3] N. N. Quadri, H. Choudhary, N. Ahmad, J. Alqahtani, and A. Qahmash, "Mobile Learning in Higher Education: A Systematic Literature Review," *Sustainability*, vol. 15, p. 13566, Sep. 2023, doi: 10.3390/su151813566.

[4] J. Yeboah and G. D. Ewur, "The Impact of Whatsapp Messenger Usage on Students Performance in Tertiary Institutions in Ghana," *J. Educ. Pract.*, vol. 5, no. 6, p. 157, 2014.

[5] "LostNet: A smart way for lost and find | PLOS One." Accessed: Apr. 27, 2026. [Online]. Available: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0310998

[6] S. Y. Tan and C. R. Chong, "AN EFFECTIVE LOST AND FOUND SYSTEM IN UNIVERSITY CAMPUS," *J. Inf. Syst. Technol. Manag.*, vol. 8, no. 32, pp. 99–112, Sep. 2023, doi: 10.35631/JISTM.832007.

[7] J. A. Schafer, E. Heiple, M. J. Giblin, and G. W. Burruss, "Critical incident preparedness and response on post-secondary campuses," *J. Crim. Justice*, vol. 38, no. 3, pp. 311–317, May 2010, doi: 10.1016/j.jcrimjus.2010.03.005.

[8] A. Howard *et al.*, "Searching for MobileNetV3," arXiv.org. Accessed: Apr. 27, 2026. [Online]. Available: https://arxiv.org/abs/1905.02244v5

[9] N. Passalis and A. Tefas, "Deep Supervised Hashing leveraging Quadratic Spherical Mutual Information for Content-based Image Retrieval," arXiv.org. Accessed: Apr. 27, 2026. [Online]. Available: https://arxiv.org/abs/1901.05135v1

[9] "TensorFlow, 'TensorFlow Lite: Deploy machine learning models on mobile and IoT devices,' . Available: https://www.tensorflow.org/lite. - Google Search." Accessed: Apr. 27, 2026. [Online].

[11] "Firebase Documentation." Accessed: Apr. 27, 2026. [Online]. Available: https://firebase.google.com/docs

[12] “Manifesto for Agile Software Development.” Accessed: Apr. 27, 2026. [Online]. Available: https://agilemanifesto.org/

[13] “Cloudinary Image & Video API Platform - Docs Home Page | Documentation.” Accessed: Apr. 27, 2026. [Online]. Available: https://cloudinary.com/documentation

[14] M. Abadi *et al.*, “TensorFlow: A System for Large-Scale Machine Learning”.

[15] A. Howard *et al.*, “Searching for MobileNetV3,” in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, Seoul, Korea (South): IEEE, Oct. 2019, pp. 1314–1324. doi: 10.1109/ICCV.2019.00140.

[16] “Flutter documentation.” Accessed: Apr. 27, 2026. [Online]. Available: https://docs.flutter.dev/

[17] “OneSignal Documentation,” OneSignal. Accessed: Apr. 27, 2026. [Online]. Available: https://documentation.onesignal.com/docs/en/home

[18] R. Pressman and B. Maxim, *Software Engineering: A Practitioner’s Approach, 8th Ed*. 2014.

[19] “Software Engineering 10th Edition PDF Download | PDF.” Accessed: Apr. 27, 2026. [Online]. Available: https://www.scribd.com/document/687290290/Software-Engineering-10th-Edition

[20] “IEEE Recommended Practice For Software Design Descriptions - IEEE Std 1016-1998.” Accessed: Apr. 27, 2026. [Online]. Available: https://cs.bilkent.edu.tr/~cagatay/cs413/1016-1998_00741934.pdf